
From Residuals to Reasons: LLM-Guided Mechanism Inference from Tabular Data

Mohammad R. Rezaei

Department of Computer Science
University of Toronto
Vector Institute
Toronto, ON, Canada
mr.rezaei@mail.utoronto.ca

Rahul G. Krishnan

Department of Computer Science
University of Toronto
Vector Institute
Toronto, ON, Canada
rahulgk@cs.toronto.edu

Abstract

A persistent challenge in machine learning for scientific applications is jointly achieving prediction and understanding. Statistical models excel on structured data but operate as black boxes, while existing interpretability methods are largely *inspective*: they answer “which features matter?” but do not articulate how features interact or refine explanations iteratively alongside human understanding. End-to-end LLM agents must search the full output domain; we instead target the smaller, structured space of base-model residuals. We introduce Multi-Agent Residual In-Context Learning (MARICL), an agentic framework in which LLM agents analyze where a base-model fails, hypothesize missing structure from high-residual examples provided in context, and produce explicit correction terms refined through multi-turn textual gradient optimization. Across nine benchmarks spanning scientific, biomedical, socioeconomic, and synthetic settings, MARICL improves consistently over its base model on all datasets. A cross-plate transfer experiment on the Cell-Free Protein dataset (where each *plate* is an independent experimental batch with specific reagent concentrations) further validates that the inferred formulas capture real structure rather than batch-specific noise. Correction formulas frozen from one plate and applied directly to separate plates (no retraining, no LLM calls) improve predictions in over 92% of cases within the same reagent cohort, but fail systematically on plates from a different reagent regime — evidence of mechanistic generalization rather than overfitting.

1 Introduction

Tabular machine learning faces a recurring tradeoff between accuracy and interpretability. Gradient boosting and related methods are highly accurate on tabular data [Grinsztajn et al., 2022, McElfresh et al., 2024] but expose nothing about the relationships they have learned. Post hoc methods address this only partially: SHAP [Lundberg and Lee, 2017] attributes a prediction to its features and TreeSHAP extends this to pairwise interactions [Lundberg et al., 2020], but the attributions are per-sample weights on a fixed model, not a global formula that can be inspected, edited, or improved. Inherently interpretable models such as GAMs [Hastie and Tibshirani, 1987] and EBMs [Nori et al., 2019] are competitive on tabular benchmarks, but they fit smooth per-feature shape functions (and pairwise terms in GA^2Ms) rather than named symbolic expressions tied to specific feature combinations and coefficients. Symbolic regression [Cranmer, 2023, Shojaei et al., 2025] produces equations, but it fits the target end-to-end rather than diagnosing where and why a given model fails.

Recent work on *agentic interpretability* [Kim et al., 2025] argues that LLM agents can go beyond one-shot inspection by reasoning across multiple turns to refine explanations. The practical difficulty is that the loop has too many directions to explore and rarely converges on something testable unless

it is constrained. We ask: how can an agentic loop produce both accurate predictions and inspectable explanations under such a constraint?

Table 1: MARICL vs. LLM-ICL on the cell-free protein example (NAD= 0.8, sperm= 0.7, fol= 0.3, $y = 0.72$, base $\hat{y}_{\text{ML}} = 0.58$). MARICL targets only the residual; LLM-ICL predicts y from scratch.

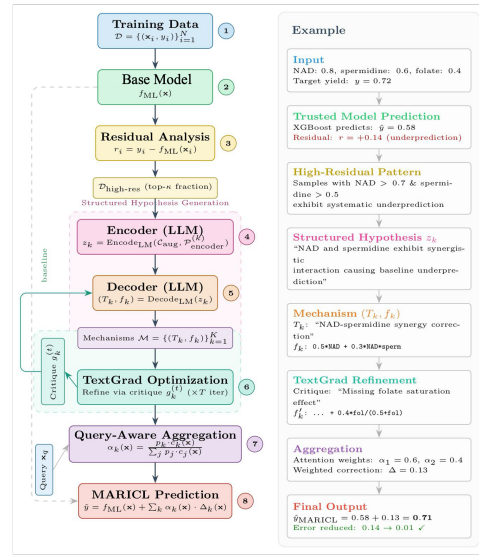
Stage	MARICL	LLM-ICL
Residual analysis	High-error samples cluster at NAD > 0.6, sperm > 0.4; feeds encoder.	—no base to compare against
Hypothesis	“NAD–spermidine cofactor synergy drives underprediction.”	—implicit in one forward pass
Formula $t=0$	$f^{(0)} = 0.5 \cdot \text{NAD} \times \text{sperm} = 0.28$	—no executable correction
Critique $t=0$	“Folinic acid saturates; add Michaelis–Menten term.”	—no feedback loop
Formula $t=1$	$f^{(1)} = f^{(0)} + \frac{0.5 \cdot \text{fol}}{0.5 + \text{fol}} = 0.4675$	—no refinement
Prediction	$\hat{y} = 0.58 + 0.28 \times 0.4675 = 0.711$ error ≈ 0.009	$\hat{y} \approx 0.52$ error ≈ 0.20 ($\sim 22\times$ larger)

A natural baseline is to give all training data to a single LLM and ask for a prediction in one pass (LLM-ICL). On cell-free protein yield prediction, this reaches only $R^2=0.35$ — worse than ordinary linear regression. The lesson is that the LLM is being asked to do too much at once: regress, identify structure, and ground itself in the data, all in one forward pass. We instead use the linear model as scaffolding and ask the LLM the narrower question: *what is the linear model missing?* This gives us two design choices: (i) keep a trusted *base-model* as an anchor on the predictive task, and (ii) restrict the LLM to explaining the residual signal the base-model leaves behind.

These two choices define Multi-Agent Residual In-Context Learning (MARICL) (Figure 1). An *encoder-agent* reads high-residual training examples and produces structured hypotheses. A *decoder-agent* compiles each hypothesis into an executable correction term — a named formula over specific features. Textual gradient optimization [Yuksekgonul et al., 2024] then refines each correction by critiquing where it still fails. We run K such agents in parallel and aggregate them with performance-weighted ensembling.

The cell-free protein example illustrates how this plays out in practice. *Cell-free protein synthesis* is a biochemistry technique that produces proteins in a test tube using a cell extract plus added reagents (energy substrates, cofactors, polyamines); the prediction task is yield as a function of reagent concentrations. On the sample in Table 1, the base model underpredicts yield when NAD (an energy cofactor) and spermidine (a polyamine that boosts translation) are both high. MARICL infers cofactor synergy, generates the interaction term $\text{NAD} \times \text{sperm}$, then refines it with a saturation term for folinic acid (which has diminishing returns). Across the dataset this lifts R^2 from 0.35 to 0.65 (+0.30 over LLM-ICL; full trace in Appendix A).

On nine benchmarks spanning scientific, biomedical, socioeconomic, and synthetic domains, MARICL improves over its base model on every dataset. Gains are largest where the base is weakest (e.g.,



+0.236 ΔR^2 over a linear base on Cell-Free Protein) and smaller but consistent over stronger bases such as XGBoost.

Contributions. (1) MARICL: an agentic framework that produces named, executable correction terms over a base model through structured hypothesis generation and iterative refinement. (2) A residual-conditioned adaptation of textual-gradient optimization [Yuksekgonul et al., 2024], paired with a distance-based aggregation that down-weights each correction when applied far from the residuals it was inferred from. (3) A layered ablation that progressively removes the LLM’s pretraining priors (feature names, domain context, frontier-model capability), together with a planted-ground-truth synthetic benchmark no LLM has seen. Together these lower-bound the data-driven share of MARICL’s gain at $\sim 50\%$ on real benchmarks. (4) A cross-plate transfer experiment on Cell-Free Protein, where each *plate* is one experimental batch. Correction formulas frozen on one plate and applied verbatim to other plates improve over 92% of pairs within the same reagent protocol and fail systematically across protocols — evidence that the corrections capture biochemical mechanism rather than batch-specific noise.

2 Methods

We consider supervised learning on $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$ (regression) or $y_i \in \{1, \dots, C\}$ (classification), split into train, validation, and test sets. We assume access to a pretrained base model $f_{\text{ML}} : \mathbb{R}^d \rightarrow \mathcal{Y}$ — linear, tree, or gradient-boosted. MARICL learns K correction agents whose predictions are added on top of the base model; interpretability lives in the corrections, which name what the base model systematically misses.

Each agent produces a correction $m_k = (T_k, f_k)$ that pairs a natural-language template T_k with a closed-form formula f_k . For regression, $f_k : \mathbb{R}^d \rightarrow \mathbb{R}$ (clipped to the scaled target range; Appendix N.3). For classification, $f_k : \mathbb{R}^d \rightarrow \mathbb{R}^C$ produces per-class scores converted to a distribution Q_k in Section 2.3. The MARICL prediction is then:

$$\hat{y}_{\text{MARICL}}(\mathbf{x}) = f_{\text{ML}}(\mathbf{x}) + \sum_{k=1}^K \alpha_k(\mathbf{x}) \cdot \Delta_k(\mathbf{x}) \quad (1)$$

where $\Delta_k(\mathbf{x}) = f_k(\mathbf{x})$ and $\alpha_k(\mathbf{x}) \geq 0$, $\sum_{k=1}^K \alpha_k(\mathbf{x}) = 1$ are query-dependent attention weights (Section 2.4). For classification:

$$P_{\text{MARICL}}(\mathbf{x}) = \beta \cdot P_{\text{ML}}(\mathbf{x}) + (1 - \beta) \sum_{k=1}^K \alpha_k(\mathbf{x}) \cdot Q_k(\mathbf{x}) \quad (2)$$

with $P_{\text{ML}}(\mathbf{x}), Q_k(\mathbf{x}) \in \Delta^{C-1}$ and $\beta \in [0, 1]$.

Residual analysis. For each training example we compute residuals

$$r_i = \begin{cases} y_i - f_{\text{ML}}(\mathbf{x}_i) & \text{(regression)} \\ \mathbb{I}[f_{\text{ML}}(\mathbf{x}_i) \neq y_i] \cdot (1 - P_{\text{ML}}(\mathbf{x}_i)_{y_i}) & \text{(classification)} \end{cases} \quad (3)$$

and select the top- κ fraction by $|r_i|$:

$$\mathcal{D}_{\text{high-res}} = \{(\mathbf{x}_{\pi(i)}, y_{\pi(i)}, r_{\pi(i)}) : i \leq \lfloor \kappa N_{\text{train}} \rfloor\}, \quad (4)$$

where π orders examples by descending $|r_i|$. A step-by-step trace of the full pipeline on a concrete Cell-Free Protein example appears in Appendix A.

2.1 Structured Hypothesis Generation

Rather than prompting an LLM directly for formulas, MARICL uses an encoder–decoder with a structured representation z_k to hypothesize explanations for the failures of the base-model.

Encoder: from residuals to structured hypotheses. We construct an augmented context

$$\mathcal{C}_{\text{aug}} = (\mathcal{D}_{\text{high-res}}, \mathcal{C}_{\text{domain}}, \mathcal{C}_{\text{features}}) \quad (5)$$

combining high-residual examples, optional domain context, and feature descriptions. The encoder analyses residual patterns:

$$z_k = \text{Encode}_{\text{LM}}(\mathcal{C}_{\text{aug}}, \mathcal{P}_{\text{encoder}}^{(k)}), \quad (6)$$

where each $\mathcal{P}_{\text{encoder}}^{(k)}$ targets a different aspect: (1) *error patterns* — which feature combinations drive high errors and what nonlinearities the base-model misses; (2) *sample patterns* — direct feature–target relationships visible in high-residual examples.

Batched encoding. When $|\mathcal{D}_{\text{high-res}}| > B$, we partition into batches, encode independently, and concatenate to preserve all insights:

$$z_{k,b} = \text{Encode}_{\text{LM}}(\mathcal{C}_{\text{aug}}^{(b)}, \mathcal{P}_{\text{encoder}}^{(k)}), \quad z_k = \text{Concat}(z_{k,1}, \dots, z_{k, \lceil |\mathcal{D}_{\text{high-res}}|/B \rceil}). \quad (7)$$

Decoder: from hypotheses to executable corrections. The decoder transforms z_k into a correction $(T_k, f_k) = \text{Decode}_{\text{LM}}(z_k, \mathcal{P}_{\text{decoder}})$ comprising a natural-language explanation T_k and a Python expression f_k . This separation forces the LLM to articulate *why* errors occur before specifying *how* to correct them, while diverse $\mathcal{P}_{\text{encoder}}^{(k)}$ encourage complementary hypotheses.

2.2 Correction Refinement via Textual Gradient Optimization

Each correction maintains a textual state $\mathcal{S}_k^{(t)}$ comprising the hypothesis $z_k^{(t)}$, correction $(T_k^{(t)}, f_k^{(t)})$, and accumulated critique history. At $t=0$:

$$z_k^{(0)} = \text{Encode}_{\text{LM}}(\mathcal{C}_{\text{aug}}, \mathcal{P}_{\text{encoder}}^{(k)}), \quad (T_k^{(0)}, f_k^{(0)}) = \text{Decode}_{\text{LM}}(z_k^{(0)}, \mathcal{P}_{\text{decoder}}). \quad (8)$$

When available, base-model knowledge (feature importances, coefficients) guides initialization. At iteration t , we evaluate the train loss

$$\mathcal{L}_k^{(t)} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\mathbf{x}, y)} \ell(f_{\text{ML}}(\mathbf{x}) + \Delta_k^{(t)}(\mathbf{x}), y) \quad (9)$$

and identify the failure set

$$\mathcal{E}_k^{(t)} = \begin{cases} \{(\mathbf{x}, y, \hat{y}) : |\hat{y} - y| > \tau_{\text{fail}}\} & \text{(regression)} \\ \{(\mathbf{x}, y, \hat{\mathbf{p}}) : \hat{p}_y < 1 - \tau_{\text{fail}}\} & \text{(classification)} \end{cases} \quad (10)$$

where $\hat{\mathbf{p}}$ is the current ensemble probability vector (Eq. 2); the classification criterion is equivalent to cross-entropy above $-\log(1 - \tau_{\text{fail}})$. We use $\tau_{\text{fail}} = 0.5$ (Appendix N.2). The per-correction loss in Eq. 9 drives selection of m_k^* , while the failure set conditions on the ensemble so that critique targets points where the full predictor still fails on the true class. A textual gradient (critique) is then generated:

$$g_k^{(t)} = \text{LM}(\mathcal{P}_{\text{critique}} \mid z_k^{(t)}, m_k^{(t)}, \mathcal{L}_k^{(t)}, \mathcal{E}_k^{(t)}), \quad (11)$$

asking the LLM to analyze why the correction fails on $\mathcal{E}_k^{(t)}$ and suggest refinements. Across both regimes, $\mathcal{E}_k^{(t)}$ identifies the queries most informative for refinement: high-error regression points and low-true-class-confidence classification points. The state accumulates

$$\mathcal{S}_k^{(t)} = \mathcal{S}_k^{(t-1)} \cup \{(z_k^{(t)}, m_k^{(t)}, \mathcal{L}_k^{(t)}, g_k^{(t)})\}, \quad (12)$$

and subsequent generations condition on the full history:

$$(T_k^{(t+1)}, f_k^{(t+1)}) \sim p_{\text{LM}}(\cdot \mid \mathcal{C}_{\text{aug}}, \mathcal{S}_k^{(t)}, \mathcal{P}_{\text{decoder}}). \quad (13)$$

Notation. The hypothesis $z_k = z_k^{(0)}$ is produced once (possibly batched via Eq. 7 with batch-shard subscript $z_{k,b}$) and held fixed; only $(T_k^{(t)}, f_k^{(t)})$ change across iterations t . The superscript (t) on $z_k^{(t)}$ in Eqs. 11–13 indexes the archived tuple within the state $\mathcal{S}_k^{(t)}$, not a re-encoded hypothesis at iteration t . We run T iterations and retain $m_k^* = \arg \min_{t \in [T]} \mathcal{L}_k^{(t)}$.

2.3 Probability Conversion for Classification

For classification, f_k outputs per-class scores $\mathbf{s}_k(\mathbf{x}) = [s_k^{(1)}(\mathbf{x}), \dots, s_k^{(C)}(\mathbf{x})]$, converted via temperature-scaled softmax:

$$Q_k(\mathbf{x})_c = \frac{\exp(s_k^{(c)}(\mathbf{x})/\tau_k)}{\sum_{c'} \exp(s_k^{(c')}(\mathbf{x})/\tau_k)}, \quad (14)$$

with τ_k chosen by validation (ECE; Appendix N.2). Query-specific confidence is

$$c_k(\mathbf{x}) = \sigma\left(\gamma \cdot (1 - d_k(\mathbf{x}))\right), \quad (15)$$

where $d_k(\mathbf{x}) = \min(\tilde{d}_k(\mathbf{x})/D_k^{95}, 1)$, $\tilde{d}_k(\mathbf{x}) = \min_{(\mathbf{x}', \cdot) \in \mathcal{D}_{\text{high-res}}^{(k)}} \|\mathbf{x} - \mathbf{x}'\|_2$ is computed in standardized feature space (frozen train statistics), D_k^{95} is the 95th percentile of pairwise distances within $\mathcal{D}_{\text{high-res}}^{(k)}$, and clipping ensures $d_k \in [0, 1]$ on far queries.

2.4 Query-Aware Aggregation

Training-time example selection. During hypothesis generation, informative examples are selected from the high-residual pool via

$$s(\mathbf{x}_q, \mathbf{x}_i) = \exp(-\|\mathbf{x}_q - \mathbf{x}_i\|_2^2 / 2\sigma^2) \cdot |r_i|^{\gamma_s}, \quad (16)$$

where the *query anchor* $\mathbf{x}_q$ is the unselected high-residual example with the largest $|r_i|$ remaining in the pool (greedy top-residual seed; ties broken by index), σ is the median pairwise distance over the pool, and $\gamma_s > 0$ trades off residual magnitude against spatial proximity (distinct from γ in Eq. 15). The score rewards examples that are both large in residual and near the anchor, yielding a locally representative, correctively informative context. This is used *only during training*; inference uses Eq. 18. **Inference-time aggregation (zero LLM cost).** At inference, each compiled f_k is executed directly. Corrections are weighted by a global performance score

$$p_k = \begin{cases} \exp(-\text{MAE}_k/\tau) & (\text{regression}) \\ \text{FI}_k^{\text{macro}} & (\text{classification}) \end{cases} \quad (17)$$

where $\tau > 0$ scales MAE_k (fixed vs. target range; Appendix N.2). Scores p_k use $\mathcal{D}_{\text{train}}$; β , τ_k , K , κ use validation. Combined with $c_k(\mathbf{x})$ from Eq. 15:

$$\alpha_k(\mathbf{x}) = \frac{p_k \cdot c_k(\mathbf{x}) \cdot \mathbb{I}[p_k > p_{\min}]}{Z(\mathbf{x})},$$

$$Z(\mathbf{x}) = \sum_j p_j \cdot c_j(\mathbf{x}) \cdot \mathbb{I}[p_j > p_{\min}], \quad (18)$$

where $p_{\min}$ filters unreliable corrections. If $Z(\mathbf{x}) = 0$ (all corrections fail the threshold), MARICL falls back to the base-model prediction. The complete inference pipeline involves only arithmetic and nearest-neighbor lookups with zero LLM overhead. Algorithm 1 presents the training procedure.

Algorithm 1 MARICL Training Algorithm

- 1: **Input:** Training data $\mathcal{D}_{\text{train}}$, base-model f_{ML} , number of corrections K , refinement iterations T , residual fraction κ , performance threshold $p_{\min}$, batch size B
 - 2: **Output:** Correction ensemble $\mathcal{M}^*$, performance scores $\{p_k\}$
 - 3: Compute residuals r_i for all $(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{train}}$ via Eq. 3
 - 4: Construct $\mathcal{D}_{\text{high-res}}$ as top- κ fraction by $|r_i|$
 - 5: Build augmented context $\mathcal{C}_{\text{aug}}$ (Eq. 5)
 - 6: **for** $k = 1$ **to** K **do**
 - 7: **if** $|\mathcal{D}_{\text{high-res}}| > B$ **then**
 - 8: Partition into batches; encode each via Eq. 7
 - 9: $z_k^{(0)} \leftarrow \text{Concat}(z_{k,1}, z_{k,2}, \dots)$
 - 10: **else**
 - 11: $z_k^{(0)} \leftarrow \text{Encode}_{\text{LM}}(\mathcal{C}_{\text{aug}}, \mathcal{P}_{\text{encoder}}^{(k)})$
 - 12: **end if**
 - 13: $(T_k^{(0)}, f_k^{(0)}) \leftarrow \text{Decode}_{\text{LM}}(z_k^{(0)}, \mathcal{P}_{\text{decoder}})$
 - 14: Validate $f_k^{(0)}$; regenerate if invalid
 - 15: $\mathcal{S}_k^{(0)} \leftarrow \{(z_k^{(0)}, m_k^{(0)})\}$
 - 16: **end for**
 - 17: **for** $k = 1$ **to** K **do**
 - 18: **for** $t = 0$ **to** $T - 1$ **do**
 - 19: Evaluate $\mathcal{L}_k^{(t)}$ on $\mathcal{D}_{\text{train}}$ using $(T_k^{(t)}, f_k^{(t)})$
 - 20: Identify failure set $\mathcal{E}_k^{(t)}$ (Eq. 10)
 - 21: Generate critique $g_k^{(t)}$ (Eq. 11)
 - 22: $\mathcal{S}_k^{(t+1)} \leftarrow \mathcal{S}_k^{(t)} \cup \{(z_k^{(t)}, m_k^{(t)}, \mathcal{L}_k^{(t)}, g_k^{(t)})\}$
 - 23: Refine: $(T_k^{(t+1)}, f_k^{(t+1)})$ via Eq. 13
 - 24: Validate $f_k^{(t+1)}$; regenerate if invalid
 - 25: **end for**
 - 26: $m_k^* \leftarrow \arg \min_t \mathcal{L}_k^{(t)}$; compute p_k (Eq. 17)
 - 27: **end for**
 - 28: **return** $\mathcal{M}^* = \{m_k^* : p_k > p_{\min}\}, \{p_k\}$
-

3 Related Work

Interpretable ML. Post-hoc methods like SHAP [Lundberg and Lee, 2017] (including TreeSHAP and the Shapley interaction index [Lundberg et al., 2020]) and LIME [Ribeiro et al., 2016] attribute predictions to features and pairs of features, but the attributions are local, per-sample, and tied to a fixed predictor; they cannot articulate *why* a model fails as a closed-form formula and provide no mechanism for iteratively refining it. Inherently interpretable models like GAMs [Hastie and Tibshirani, 1987] and EBM [Nori et al., 2019] fit smooth nonparametric shape functions per feature, with optional pairwise terms in GA^2Ms [Lou et al., 2013]; we compare against *EBM (with pairwise)* as a strong interpretable interaction baseline (Table 7). MARICL is complementary: it produces explicit, closed-form symbolic formulas (named cofactor products, saturation terms, sigmoidal gates) rather than per-pair learned response surfaces, and refines them iteratively against base-model failures. InterpreTabNet [Si et al., 2024] learns sparse attention masks over features and uses LLMs to generate natural language explanations of feature interdependencies, but remains inspective rather than iteratively refining hypotheses. MARICL addresses a complementary question: where does a model systematically fail and why?

Symbolic Regression. Classical genetic programming [Cranmer, 2023] and LLM-guided methods [Shojaee et al., 2025] discover closed-form expressions from data. LLM-LEx [Harvey et al., 2025] uses LLMs to generate predictive formulas directly. These methods focus on fitting data globally; MARICL instead targets residual patterns through iterative hypothesis refinement, discovering equations that explain *where and why* baselines err.

Multi-Agent and Textual Optimization. Debate frameworks [Du et al., 2023] and mixture-of-agents [Wang et al., 2024] coordinate LLMs on shared tasks. TextGrad [Yuksekgonul et al., 2024] and self-refine [Madaan et al., 2023] optimize outputs through critique. MARICL differs: each agent specializes on distinct failure patterns via residual analysis, and multi-turn refinement targets interpretable formulas validated by predictive performance.

In-Context Learning, Boosting, and LLM Feature Engineering. Demonstration selection methods leverage similarity [Liu et al., 2021] or diversity [Levy et al., 2023]. Gradient boosting [Friedman, 2001] fits weak learners to residuals. CAAFE [Hollmann et al., 2023a] prompts an LLM to generate executable feature-engineering code on the input space, using dataset descriptions as context and validating each candidate against held-out accuracy. MARICL shares the compile-to-executable strategy but operates on the *residual* space rather than the input space, generating corrections through iterative TextGrad refinement rather than one-shot generation, and producing named symbolic terms tied to specific feature combinations rather than augmented feature columns; in combination, this provides both executable corrections and human-readable hypotheses about feature interactions causing failures.

4 Experiments

We evaluate MARICL on nine benchmarks across scientific, biomedical, social, economic, and synthetic domains (mean $\pm$ std across random seeds). Four questions structure the evaluation: (i) does MARICL improve prediction; (ii) can it recover known structure from data; (iii) how much of the gain comes from iterative refinement versus the LLM’s pretraining priors; and (iv) do the learned formulas generalize beyond their training data?

Datasets. Five regression tasks (*Cell-Free Protein Production* [Borkowski et al., 2020], *Enzyme Activity*, *Diabetes Progression*, *California Housing*, *Bike Sharing*) and three classification tasks (*Zoo*, *High School Social Classification*, *Adult Income*). We additionally construct a *Synthetic Benchmark* ($N = 1,000$, $d = 8$) with a planted ground-truth formula that cannot appear in any LLM pretraining corpus (§4.2). **Baselines.** Linear/Logistic Regression, XGBoost, EBM [Nori et al., 2019], SHAP [Lundberg and Lee, 2017], TabPFN [Hollmann et al., 2023b], LLM-LEx [Harvey et al., 2025], LLM-ICL, Symbolic Regression [Cranmer, 2023], and PySR-on-residuals. Cross-validation, hyperparameter sensitivity, calibration, and paired Wilcoxon tests with Benjamini–Hochberg FDR control appear in Appendices N and K.

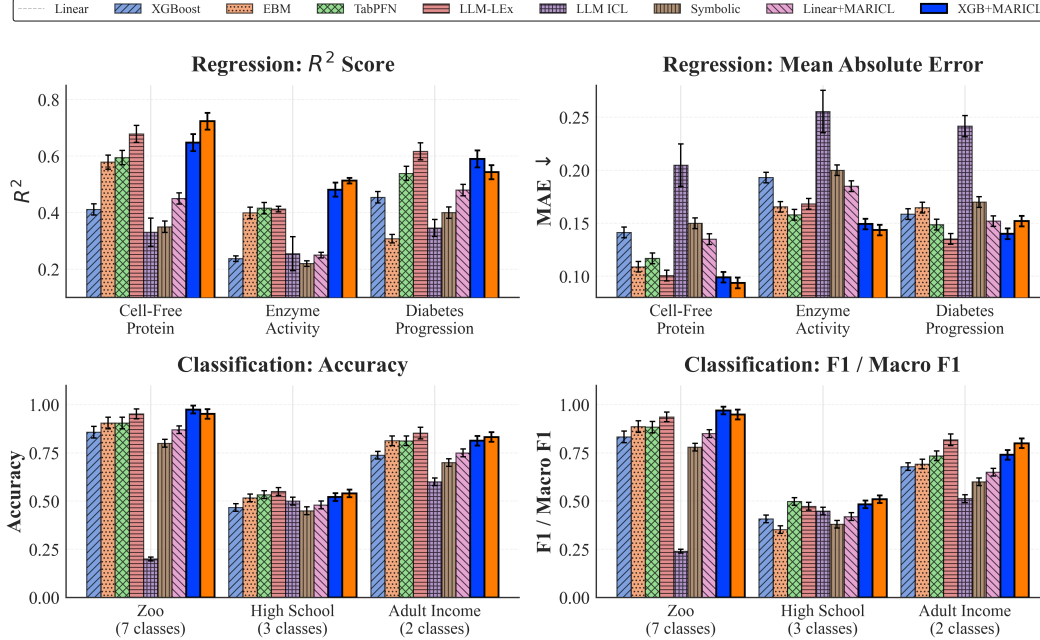


Figure 2: Performance across regression and classification benchmarks (± 1 std, 5 seeds). Full numerical results in Tables 16, 15.

4.1 MARICL Improves Prediction on Eight Real Benchmarks

MARICL outperforms all base models on every real benchmark, lifting R^2 and macro-F1 over both Linear and XGBoost bases (Figure 2; Tables 15, 16). Within-base gains are largest when the base is weak, since a stronger base leaves less structured residual to correct. Against TabPFN the gap falls within one standard deviation on the hardest tasks, but MARICL additionally returns a human-readable formula. LLM-Lex and LLM-ICL both underperform, confirming that the iterative refinement loop is necessary; single-pass generation is not enough.

The Diabetes task illustrates how the choice of base matters: Linear+MARICL outperforms XGBoost+MARICL because the linear base is itself stronger on this dataset, and MARICL inherits that advantage. On Adult Income, macro-F1 rises from 0.692 to 0.800 (+0.108, +15.6%) over XGBoost; the XGBoost baseline has poor minority-class recall, and MARICL’s corrections concentrate on those high-residual examples. Even with XGBoost tuned over a five-axis grid (Table 25), MARICL leads by +0.106 R^2 on Cell-Free Protein and +0.069 macro-F1 on Adult Income, with positive gaps on all nine benchmarks. Among residual-correction baselines (PySR-on-residuals, pairwise interactions, EBM, MLP-on-residuals), MARICL is the only method that achieves both the best within-model R^2 and a human-readable formula (Appendix B).

4.2 Synthetic Benchmark: Recovering Planted Ground Truth

Gains on real benchmarks could in principle come from the LLM recognising the domain rather than reasoning over the data. To rule this out we construct a benchmark with a known ground truth that the LLM cannot have seen:

Table 2: Synthetic benchmark with planted ground truth. MARICL recovers the dominant sigmoid as an explicit symbolic term. XGBoost is a standalone full model; its ΔR^2 is reported over Linear for comparability, not as a residual correction.

Method	Type	R^2	ΔR^2	Sigmoid?
Linear (base)	Standalone (base)	0.387	—	—
PySR on residuals	Residual correction	0.854	+0.467	Partial
XGBoost	Standalone	0.831	+0.444	No
MARICL (ours)	Residual correction	0.892	+0.505	Yes
Oracle correction	Standalone (oracle)	0.961	+0.574	—

$$Y = 0.6X_1 + 0.4X_2 + 2.5\sigma(1.8X_1X_3 - 1.2) + 0.3\sin(X_5X_7) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 0.1^2)$$

Table 3: Interpretable corrections for protein expression and zoo classification. “Correction Only” = learned correction without the base-model; “MARICL” = full combined performance.

Dataset	Learned Correction Formula	Domain Interpretation & Literature Support	Correction Only	MARICL
Protein Expression	$\hat{y} = 1.5 \cdot \text{NAD} + 1.2 \cdot \text{spermidine} + 0.5 \cdot \text{NAD} \times \text{spermidine} + 0.6 \cdot \text{CoA} + 0.5 \cdot 3\text{-PGA} + \frac{0.5 \cdot \text{folinic_acid}}{0.5 + \text{folinic_acid}}$	NAD-spermidine synergy: polyamines stimulate translation 3–5× [Atkins et al., 1975]; the PANox-SP system uses NAD and CoA to regenerate ATP [Jewett and Swartz, 2004]. Folinic acid saturation: formyl-methionine precursor with an empirical concentration plateau [Cai et al., 2015].	0.495	0.723 R^2
Zoo Animals	$\text{score}_{\text{mammal}} = 1.1 \cdot \text{hair} + 1.3 \cdot \text{milk} + 0.8 \cdot (\text{hair} \times \text{milk} \times (1 - \text{eggs}))$ $\text{score}_{\text{bird}} = 1.2 \cdot \text{feathers} + 0.8 \cdot \text{airborne} + 0.5 \cdot (\text{feathers} \times \text{airborne})$ $\text{score}_{\text{fish}} = 1.0 \cdot \text{fins} + 0.9 \cdot \text{aquatic} + 0.4 \cdot (\text{fins} \times \text{aquatic} \times \text{backbone})$	Taxonomic encoding: $\text{hair} \times \text{milk} \times (1 - \text{eggs})$ captures mammalian synapomorphies. Feathers over flight: prioritizes morphology over behavior so that flightless birds remain birds. <i>Treated as controlled validation; LLM priors on taxonomy likely contribute.</i>	0.762	0.975 Acc

with iid features ($N=1,000$, $d=8$; Appendix D). A linear base captures the additive terms but misses the sigmoid nonlinearity, which is therefore the structure a residual correction must recover.

Table 2 shows that MARICL recovers the planted sigmoid as the explicit symbolic term $c \cdot \sigma(a X_1 X_3 - b)$, with $(a, b) = (1.6, 1.1)$ within $\sim 12\%$ of the true $(1.8, 1.2)$; the sin term is absorbed into polynomial residuals and the amplitude c is folded into linear coefficients (a limitation we flag in §5). PySR-on-residuals reaches comparable R^2 but returns a 12-node nested expression that obscures the planted structure even when σ is provided as a primitive (Appendix N.5). Because the planted mechanism cannot exist in pretraining data, this experiment isolates the data-driven component of MARICL’s performance.

4.3 Disentangling Data-Driven Refinement from Prior Knowledge

How much of MARICL’s improvement comes from the iterative loop, and how much from the LLM already knowing the domain (e.g., that NAD is a translation cofactor or that BMI predicts diabetes)? We separate the two by progressively removing what the LLM gets “for free” from pretraining.

(i) *Removing the domain prompt* drops performance by only 2.3–3.5 points and still recovers the same dominant correction category (Appendix C). (ii) *Anonymizing feature names* (replacing them with opaque `feat_0`, ..., `feat_d`) preserves 67–69% of MARICL’s gain on Cell-Free Protein and Diabetes (Appendix I). (iii) *Disabling refinement* ($T=0$): performance increases monotonically with the number of iterations (Table 5), so any gain above $T=0$ is attributable to the loop rather than the initial prompt. (iv) *All three at once*: with feature names anonymized, no domain context anywhere in the prompts, and a small open LLM (Llama-3-8B) replacing frontier models, roughly half of MARICL’s end-to-end gain still survives — and still exceeds PySR-on-residuals (Appendix L).

This puts a lower bound of $\sim 50\%$ on the share of MARICL’s gain attributable to data-driven refinement on real benchmarks; the remainder is the deployment benefit when the LLM’s pretraining happens to align with the domain. The synthetic benchmark (§4.2) and cross-plate transfer (§4.5) test the data-driven half directly, since neither admits a pretraining shortcut.

4.4 Interpretability of Learned Corrections

MARICL’s corrections are explicit symbolic formulas that name what the base model misses (Table 3). For Cell-Free Protein, the correction encodes NAD–spermidine cofactor synergy (polyamines stimulate translation 3–5× [Atkins et al., 1975]; NAD and CoA are ATP-regeneration cofactors [Jewett and Swartz, 2004]) and a Michaelis–Menten saturation term for folinic acid [Cai et al., 2015]. The “Correction Only” column shows that the formula carries meaningful structure on its own, independent of the base model. For Zoo classification, the correction encodes $\text{hair} \times \text{milk} \times (1 - \text{eggs})$ for mammalian synapomorphies and weights feathers above flight ability so that flightless birds remain classified as birds; LLM priors on taxonomy plausibly contribute here, so we treat this as controlled validation rather than a data-driven inference. When a correction is unreliable, the $p_{\min}$ threshold (Eq. 18) suppresses it.

4.5 Frozen Correction Formulas Transfer Across Plates Without Retraining

Setup. We use the Cell-Free Protein dataset, in which proteins are produced in a test tube from a cell extract plus a mixture of reagents (energy substrates, cofactors, polyamines, etc.). Each *plate* is one experimental batch, run with a specific reagent recipe, and yields measurements for many wells. The dataset’s plates were collected over several rounds of the experiment. Plates AL_2 through AL_6 share one reagent protocol; AL_7 through AL_10 used a different protocol with different reagent ranges. We refer to these two groups as *reagent cohorts*.

Hypothesis. If MARICL’s corrections capture real biochemistry rather than batch-specific noise, then a formula learned on one plate should still improve predictions on a different plate from the same cohort, with no retraining and no LLM calls. Held-out accuracy on the same plate is too weak a check (formulas overfit to a single batch can still generalize within that batch); cross-plate transfer is the harder test.

Result. We freeze correction formulas from 37 source runs on plate AL_6 and apply them to 8 target plates without any updates and without any LLM calls (Appendix E.1). Within the same reagent cohort (AL_2–AL_5), the frozen corrections improve over the base model on 92–97% of source-target pairs, with mean MAE reductions of 0.155 to 0.249 per plate (Table 4). On plates from the different cohort (AL_7–AL_10), the same formulas improve only 0–8% of pairs.

Table 4: Cross-plate transfer: formulas frozen from source plate AL_6 applied to target plates with zero LLM cost. “Same reagent” plates share AL_6’s protocol; “Diff. regime” plates use a different one. Adding the corrections consistently beats the base model within the same cohort and fails across the regime boundary. Per-plate breakdown in Appendix E.

Target cohort	% improvement	Avg Δ MAE
Same reagent (AL_2–AL_5)	92–97%	+0.155 to +0.249
Diff. regime (AL_7–AL_10)	0–8%	−0.155 to −0.213

The success/failure boundary aligns exactly with the reagent-protocol boundary, not with the number of plates between source and target. This is what mechanistic generalization predicts: a formula that captures real biochemistry transfers wherever that biochemistry holds, while an overfit formula would not produce such a clean boundary. Robustness of the quality filter and the effect of the base model on transfer are in Appendix E.

4.6 Ablation Studies

Table 5 ablates the main hyperparameters on Cell-Free Protein, holding everything else at the headline configuration ($K=2$, $T=10$, Top-K= 100, scaled features). Adding a second correction ($K=1 \rightarrow 2$) yields $+0.107 \Delta R^2$ for Linear and $+0.033$ for XGBoost; the smaller XGBoost gain reflects its stronger base, which leaves less residual signal for additional corrections to absorb (§M.3). Larger residual pools (Top-K) yield monotonically richer signal. Feature scaling is essential for the linear base. More refinement iterations improve performance ($T=10 > T=5 > T=3$), confirming that the loop adds value beyond the initial prompt. Training requires 14–82 LLM calls per run (\$0.02–\$0.11 with Gemini 2.0 Flash); cost analysis and backbone robustness are in Appendices F and G.

Table 5: Ablations on Cell-Free Protein (mean $\pm$ std, 5 seeds). Each sub-block varies the named factor while holding all other factors at the headline configuration ($K=2$, $T=10$, Top-K=100, scaled features). The $K=2$ Linear (0.648) and XGBoost (0.723) entries match the headline values in Table 15.

Configuration	R^2	Config	R^2
# Corr. (Linear)		# Corr. (XGBoost)	
1 corr.	0.541 ± 0.027	1 corr.	0.690 ± 0.025
2 corr.	0.648 ± 0.030	2 corr.	0.723 ± 0.031
Top-K (XGBoost)		Feature Scaling (Linear)	
Top-K = 30	0.626 ± 0.024	Unscaled	0.412 ± 0.029
Top-K = 50	0.711 ± 0.021	Scaled	0.648 ± 0.030
Top-K = 100	0.723 ± 0.031		
Refinement Iter. (XGBoost, $K=2$, Top-K=100)			
$T = 0$ (base only)	0.5787 ± 0.025	$T = 5$	0.6789 ± 0.026
$T = 3$	0.6234 ± 0.024	$T = 10$	0.7231 ± 0.031

5 Discussion and Conclusion

MARICL deploys LLM agents to analyze a base-model’s systematic failures: an encoder transforms residual patterns into structured hypotheses, a decoder converts these into executable corrections, and textual gradient optimization iteratively refines them against predictive performance. **When MARICL succeeds.** Within-model gains are largest in domains with correctable residual structure: $+0.236 \Delta R^2$ over a Linear base on Cell-Free Protein, $+0.144$ over XGBoost on the same task. MARICL is most clearly advantageous when the practitioner’s validated base leaves substantial structured residual, or when interpretable corrections are valued alongside accuracy — the regime where end-to-end LLM agents (LLM-ICL) underperform. **Limitations.** MARICL’s gains are modest when the base already captures the dominant nonlinearity or under high-dimensional noise (§M.3); $p_{\min}$ ensures graceful degradation. Training requires a small LLM-call budget (Appendix F); inference is zero-cost. The joint prior-stripping ablation (§L) shows roughly half of MARICL’s gain on real benchmarks survives without feature semantics, domain context, or a frontier LLM; the other half is a deployment benefit when priors align with the domain. The synthetic and cross-plate results (§4.2, §4.5) carry the inference argument on tasks that admit no pretraining shortcut. Corrections may capture correlations rather than causation, and formulas are scoped to the regime they were learned in. **Broader impact.** Among the residual-correction baselines we evaluate, MARICL is the only one that jointly achieves the highest within-model R^2 and produces named, closed-form formulas suitable for domain-expert inspection — a Pareto-style claim rather than dominance over all interpretable methods. In scientific applications, understanding *where and why* models fail can generate testable hypotheses, and the cross-plate result shows that formulas can track meaningful experimental boundaries, supporting reuse across related conditions and flagging when fresh discovery is needed.

References

- Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35, pages 507–520, 2022.
- Duncan McElfresh, Sujay Khandagale, Jonathan Valverde, Vishak Prasad C, Ganesh Ramakrishnan, Micah Goldblum, and Colin White. When do neural nets outperform boosted trees on tabular data? In *Advances in Neural Information Processing Systems*, volume 36, pages 76336–76369, 2024.
- Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, pages 4765–4774, 2017.
- Scott M Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020.
- Trevor Hastie and Robert Tibshirani. Generalized additive models: some applications. *Journal of the American Statistical Association*, 82(398):371–386, 1987.
- Harsha Nori, Samuel Jenkins, Paul Koch, and Rich Caruana. InterpretML: A unified framework for machine learning interpretability. *arXiv preprint arXiv:1909.09223*, 2019.
- Miles Cranmer. Interpretable machine learning for science with pysr and symbolicregression.jl. *arXiv preprint arXiv:2305.01582*, 2023.
- Parshin Shojaei, Kazem Meidani, Shashank Gupta, Amir Barati Farimani, and Chandan K Reddy. Llm-sr: Scientific equation discovery via programming with large language models. In *International Conference on Learning Representations*, 2025. Oral presentation.
- Been Kim, John Hewitt, Neel Nanda, Noah Fiedel, and Oyvind Tafjord. Because we have llms, we can and should pursue agentic interpretability. *arXiv preprint arXiv:2506.12152*, 2025.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic differentiation via text. *arXiv preprint arXiv:2406.07496*, 2024.
- Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. “Why should I trust you?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144. ACM, 2016.

- Yin Lou, Rich Caruana, Johannes Gehrke, and Giles Hooker. Accurate intelligible models with pairwise interactions. In *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 623–631, 2013.
- Jacob Si, Wendy Yusi Cheng, Michael Cooper, and Rahul G Krishnan. InterpreTabNet: Distilling predictive signals from tabular data by salient feature interpretation. *arXiv preprint arXiv:2406.00426*, 2024.
- Thomas R Harvey, Fabian Ruehle, Kit Fraser-Taliente, and James Halverson. Symbolic regression with multimodal large language models and Kolmogorov–Arnold networks. *arXiv preprint arXiv:2505.07956*, 2025.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first International Conference on Machine Learning*, 2023.
- Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023.
- Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. What makes good in-context examples for gpt-3? arxiv preprint. *arXiv preprint arXiv:2101.06804*, 2021.
- Itay Levy, Ben Bogin, and Jonathan Berant. Diverse demonstrations improve in-context compositional generalization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1401–1422, 2023.
- Jerome H Friedman. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.
- Noah Hollmann, Samuel Müller, and Frank Hutter. Large language models for automated data science: Introducing caafe for context-aware automated feature engineering. *Advances in Neural Information Processing Systems*, 36:44753–44775, 2023a.
- Olivier Borkowski, Carlos Bricio, Michela Murgiano, Barak Rothschild-Mancinelli, Guy-Bart Stan, and Tom Ellis. Large scale active-learning-guided exploration for in vitro protein production optimization. *Nature Communications*, 11(1):1872, 2020.
- Noah Hollmann, Samuel Müller, Katharina Eggenberger, and Frank Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations*, 2023b.
- John F Atkins, James B Lewis, Carl W Anderson, and Raymond F Gesteland. Enhanced differential synthesis of proteins in a mammalian cell-free system by addition of polyamines. *Journal of Biological Chemistry*, 250(14):5688–5695, 1975.
- Michael C Jewett and James R Swartz. Mimicking the Escherichia coli cytoplasmic environment activates long-lived and efficient cell-free protein synthesis. *Biotechnology and Bioengineering*, 86(1):19–26, 2004.
- Qiong Cai, Jeffrey A Hanson, Aaron R Steiner, Cuong Tran, Mary Rose Masikat, Rishard Chen, James F Zawada, Aaron K Sato, Trevor J Hallam, and Gang Yin. A simplified and robust protocol for immunoglobulin expression in Escherichia coli cell-free protein synthesis systems. *Biotechnology Progress*, 31(3):823–831, 2015.
- Athel Cornish-Bowden. *Fundamentals of Enzyme Kinetics*. Wiley-VCH, Weinheim, 4th edition, 2012. First edition 1979; covers Michaelis-Menten kinetics, steady-state kinetics, and enzyme mechanisms.
- C Nick Pace, Bret A Shirley, Marsha McNutt, and Ketan Gajiwala. Forces contributing to the conformational stability of proteins. *The FASEB Journal*, 10(1):75–83, 1996. doi: 10.1096/fasebj.10.1.8566551.

- Ralph A DeFronzo, Ele Ferrannini, Leif Groop, Robert R Henry, William H Herman, Jens Juul Holst, Frank B Hu, C Ronald Kahn, Itamar Raz, Gerald I Shulman, Donald C Simonson, Marcia A Testa, and Ram Weiss. Type 2 diabetes mellitus. *Nature Reviews Disease Primers*, 1:15019, 2015. doi: 10.1038/nrdp.2015.19.
- Steven E Kahn, Rebecca L Hull, and Kristina M Utzschneider. Mechanisms linking obesity to insulin resistance and type 2 diabetes. *Nature*, 444(7121):840–846, 2006. doi: 10.1038/nature05482.
- Miller McPherson, Lynn Smith-Lovin, and James M Cook. Birds of a feather: Homophily in social networks. *Annual Review of Sociology*, 27(1):415–444, 2001. doi: 10.1146/annurev.soc.27.1.415.
- Daniel Kahneman and Amos Tversky. Prospect theory: An analysis of decision under risk. *Econometrica*, 47(2):263–291, 1979. doi: 10.2307/1914185.
- Dua, D. and Graff, C. UCI Machine Learning Repository: Enzyme Activity Dataset. <https://archive.ics.uci.edu/>, 2019. University of California, Irvine, School of Information and Computer Sciences.
- Bradley Efron, Trevor Hastie, Iain Johnstone, and Robert Tibshirani. Least angle regression. *Annals of Statistics*, 32(2):407–499, 2004.
- Richard Forsyth. Zoo Data Set. UCI Machine Learning Repository, <https://archive.ics.uci.edu/dataset/111/zoo>, 1990.
- UCI Machine Learning Repository. High School Student Performance / Social Classification Dataset. <https://archive.ics.uci.edu/>, 2014.
- Ron Kohavi. Scaling up the accuracy of naive-Bayes classifiers: A decision-tree hybrid. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 202–207, 1996.
- R. Kelley Pace and Ronald Barry. Sparse spatial autoregressions. *Statistics & Probability Letters*, 33(3):291–297, 1997.
- Hadi Fanaee-T and João Gama. Event labeling combining ensemble detectors and background knowledge. *Progress in Artificial Intelligence*, 2:113–127, 2014.

Contents

1	Introduction	1
2	Methods	3
2.1	Structured Hypothesis Generation	3
2.2	Correction Refinement via Textual Gradient Optimization	4
2.3	Probability Conversion for Classification	5
2.4	Query-Aware Aggregation	5
3	Related Work	6
4	Experiments	6
4.1	MARICL Improves Prediction on Eight Real Benchmarks	7
4.2	Synthetic Benchmark: Recovering Planted Ground Truth	7
4.3	Disentangling Data-Driven Refinement from Prior Knowledge	8
4.4	Interpretability of Learned Corrections	8
4.5	Frozen Correction Formulas Transfer Across Plates Without Retraining	9
4.6	Ablation Studies	9
5	Discussion and Conclusion	10
A	Worked Example: MARICL Pipeline on Cell-Free Protein Production	15
B	Symbolic Residual-Correction Baselines	16
C	Domain Context Ablation	16
D	Synthetic Benchmark: Detailed Recovery Analysis	16
E	Cross-Plate Transfer: Per-Target Breakdown	17
E.1	Implementation	18
F	Computational Cost Analysis	19
G	LLM Backbone Analysis	20
H	Additional Correction Analysis	20
I	Anonymized Features on Real Data	22
J	Complete Experimental Results	22
J.1	Decomposing Structural and Numerical Contributions	22
K	Statistical Significance	23

L	Joint Prior-Stripping Ablation	24
M	Validation Protocol and Generalization Safeguards	25
M.1	Extended Benchmarks: TabArena	25
M.2	Stability of Learned Corrections	26
M.3	Failure Modes	26
N	Extended Technical Details	26
N.1	Training Protocol and Generalization	26
N.2	Hyperparameter Configuration and Sensitivity	27
N.3	Formula Generation and Numerical Stability	28
N.4	Prompt Templates	28
N.5	Data Preprocessing and Reproducibility	30

A Worked Example: MARICL Pipeline on Cell-Free Protein Production

Table 6 traces the MARICL pipeline (Algorithm 1) on the same illustrating reaction as Figure 1: $\text{NAD}=0.8$, $\text{spermidine}=0.7$, $\text{folinic_acid}=0.3$, true yield $y = 0.72$, linear base $\hat{y}_{\text{ML}} = 0.58$, residual $r = +0.14$. The decoder stages follow the figure caption (interaction term, then folinate saturation); the interaction coefficient and Michaelis–Menten form match the corresponding terms in Table 3 (the headline correction adds further linear terms in NAD, spermidine, CoA, and 3-PGA). Numeric checks: $f_1^{(0)} = 0.5 \times 0.8 \times 0.7 = 0.28$; $f_1^{(1)} = 0.28 + 0.5 \times 0.3 / (0.5 + 0.3) = 0.4675$.

Table 6: Step-by-step trace for Figure 1, with side-by-side comparison to a single-shot LLM agent (LLM-ICL) on the same input. MARICL’s residual targeting collapses the LLM’s hypothesis space from the full output domain $[0, 1]$ to a small structured perturbation of $\hat{y}_{\text{ML}} = 0.58$. The LLM-ICL column shows an *exemplar* end-to-end prediction $\hat{y} \approx 0.52$ (error ~ 0.20 , $\sim 22 \times$ MARICL); dataset mean MAE is 0.15 with larger errors on interaction-heavy points (Table 15). Dataset-wide $+0.30 R^2$ improvement.

Pipeline Stage	MARICL Output	Single-Shot LLM Agent (LLM-ICL)
Input	Linear regression; sample: $\text{NAD}= 0.8$, $\text{spermidine}= 0.7$, $\text{folinic_acid}= 0.3$. $\hat{y}_{\text{ML}} = 0.58$; $y = 0.72$; $r = y - \hat{y}_{\text{ML}} = +0.14$.	Sees raw features ($\text{NAD}= 0.8$, $\text{sperm}= 0.7$, $\text{fol}= 0.3$) and a few in-context (input, target) demonstrations. No base anchor; must predict $y \in [0, 1]$ from scratch.
Residual Analysis (Eq. 3–4)	Top- κ training examples concentrate on $\text{NAD} > 0.6$ and $\text{spermidine} > 0.4$; residuals up to ~ 0.21 . Cofactor-rich conditions dominate the ranking; this subset feeds the encoder.	N/A — agent has no residual signal; cannot localize where the base errs.
Encoder (Eq. 5–6)	<i>“High-residual reactions share elevated NAD and spermidine. The linear model misses their interaction. Proposed mechanism: multiplicative synergy drives underprediction.”</i>	N/A — no separate hypothesis stage; structure (if any) is implicit in a single forward pass.
Decoder , (Eq. 8)	$t=0$ $T_1^{(0)}$: “NAD–spermidine cofactor synergy.” $f_1^{(0)}(\mathbf{x}) = 0.5 \cdot \text{NAD} \times \text{spermidine}$ (same interaction weight as Table 3). Here $f_1^{(0)} = 0.28$.	N/A — no executable correction term is produced.
Evaluate & Critique , $t=0$ (Eq. 10–11)	High-folonic-acid reactions still fail. $g_1^{(0)}$: <i>“Folonic acid likely saturates; add a Michaelis–Menten term.”</i>	N/A — single-pass output; no failure-driven feedback loop.
Decoder , (Eq. 13)	$t=1$ $f_1^{(1)} = 0.5 \cdot \text{NAD} \times \text{spermidine} + \frac{0.5 \cdot \text{folinic_acid}}{0.5 + \text{folinic_acid}}$ (saturation term as in Table 3). $f_1^{(1)}(\mathbf{x}) = 0.4675$; $\mathcal{L}_1^{(1)} < \mathcal{L}_1^{(0)}$.	N/A — no refinement.
Inference (Eq. 18, 1)	d_1 small $\Rightarrow$ high c_1 ; $p_1 > p_{\min}$. No LLM call. $\hat{y}_{\text{MARICL}} = 0.58 + \alpha_1 f_1^* \approx 0.58 + 0.28 \times 0.4675 = 0.711$. Error $ y - \hat{y} \approx 0.009$.	Exemplar end-to-end: $\hat{y}_{\text{LLM-ICL}} \approx 0.52$, $ y - \hat{y} \approx 0.20$ (mean test MAE 0.15; errors often larger on cofactor-rich, high-interaction samples; Table 15), $\approx 22 \times$ MARICL’s 0.009. Search space is the full $[0, 1]$ output range, not a small residual.

The contrast is not about LLM capability — both systems use the same backbone — but about *what is being predicted*: an absolute target vs. a small residual on top of a validated base.

B Symbolic Residual-Correction Baselines

This appendix expands on the baseline comparison referenced in the main text. To test whether MARICL’s gains require LLM-guided hypothesis generation rather than standard symbolic methods on residuals, we compare against PySR-on-residuals, exhaustive pairwise interactions, EBM with pairwise terms, and an MLP on residuals. PySR-on-residuals produces deeply nested 12-node expressions; the MLP is competitive on prediction but provides no interpretability. MARICL is the only method that jointly achieves the highest within-model R^2 and produces human-readable formulas (Table 7). Appendix J.1 additionally disentangles MARICL’s symbolic structure from its numerical coefficient fit by replacing LLM coefficients with least-squares values while keeping the inferred structure fixed.

Table 7: Residual-correction baselines (Linear base). MARICL achieves the highest within-model R^2 while producing named, interpretable formulas.

Method	Enzyme R^2	Diabetes R^2	Interpretable?
Linear	0.237	0.454	Yes
+ Ridge	0.298	0.489	Partial
+ PySR	0.318	0.512	No
+ Pairwise	0.362	0.531	No
+ EBM	0.416	0.539	Partial
+ MLP	0.428	0.548	No
+ Ours	0.481	0.590	Yes

C Domain Context Ablation

This appendix expands on the prior-knowledge analysis (§4.3). We replace the full domain prompt with a minimal one (feature names only). Across three datasets the same dominant correction category is recovered, and quantitative drops are bounded by 3–5 points (Table 8). Combined with the TextGrad iteration ablation (Table 5), this indicates that the iterative residual-driven loop (not the prompt) is the primary driver of MARICL’s gains.

Table 8: Domain context ablation. Removing domain-specific prompts causes only 2.3–3.5 point drops; the same dominant correction category is recovered in all cases.

Dataset	Full Cont.	Minimal Cont.
Enzyme (R^2)	0.513 ± 0.01	0.478 ± 0.018
Diabetes (R^2)	0.590 ± 0.03	0.561 ± 0.028
Zoo (Acc)	0.975 ± 0.02	0.952 ± 0.025

D Synthetic Benchmark: Detailed Recovery Analysis

This appendix expands on §4.2. The inferred MARICL sigmoid term has the form $c \cdot \sigma(a X_1 X_3 - b)$, where the recovered $(a, b) = (1.6, 1.1)$ match the planted $(1.8, 1.2)$ within $\sim 12\%$. The leading amplitude c (planted: 2.5) is not reported as a separately recovered coefficient; we conjecture it is absorbed into the upstream linear coefficients on X_1 and X_2 and the global clipping into $[0, 1]$, consistent with the overall R^2 match in Table 2. The lower-amplitude $0.3 \sin(X_5 X_7)$ term contributes roughly 6% of signal variance and is not consistently recovered as a distinct symbolic component across seeds; its contribution is absorbed into smaller polynomial residuals. Across 5 seeds, the dominant sigmoid is recovered as an explicit symbolic term in 5/5 cases, while the sin term is recovered in 1/5 cases (with high coefficient variance even there). We additionally provide per-seed coefficient estimates, R^2 on the residual after removing the sigmoid term, and a comparison of MARICL’s recovered structure against the PySR-on-residuals expression, which obscures the sigmoid inside a 12-node nested form. To ensure a fair operator-set comparison, PySR was run with σ added to its primitive set as a custom unary operator (Appendix N.5); the structural-clarity contrast above therefore reflects search-procedure differences rather than an artificially closed search space.

Feature distributions and signal-variance budget. Features $X_1, \dots, X_8$ are sampled iid from $\mathcal{U}(0, 1)$; observation noise $\varepsilon \sim \mathcal{N}(0, 0.1^2)$; sample size $N=1,000$; train/val/test split 60/20/20 with seeds $\{0, 1, 2, 3, 4\}$. Under this sampling distribution the variance budget of $Y = 0.6X_1 + 0.4X_2 + 2.5\sigma(1.8X_1X_3 - 1.2) + 0.3\sin(X_5X_7) + \varepsilon$ decomposes (numerically over 10^6 Monte-Carlo draws) as: linear additive terms ≈ 0.043 , planted sigmoid term $2.5\sigma(1.8X_1X_3 - 1.2) \approx 0.31$, sin term $0.3\sin(X_5X_7) \approx 0.018$, and noise 0.01, giving a population R^2 ceiling of ≈ 0.97 that MARICL approaches in Table 2. Reproducing these numbers without the explicit feature distribution is not possible because R^2 is sensitive to feature variance, which is why we record both the sampling distribution and the empirical signal-variance breakdown here.

E Cross-Plate Transfer: Per-Target Breakdown

This appendix expands on §4.5 (Table 4) with the full per-target breakdown (Table 9) and the four ablation configurations referenced in the main text (Table 10).

Table 9: Cross-plate transfer per target. Formulas frozen from source plate AL_6 evaluated numerically on each of 8 remaining plates. The Avg ΔR^2 column is reported in percentage points ($\times 100$) to keep its scale aligned with ΔMAE ; raw ΔR^2 is bounded above by 1.

Target	Cohort	% Impr.	Avg ΔMAE	Avg ΔR^2 (%)
AL_2	Same reagent	97%	+0.158	+6.17
AL_3	Same reagent	97%	+0.249	+9.39
AL_4	Same reagent	92%	+0.155	+8.05
AL_5	Same reagent	95%	+0.189	+10.83
AL_7	Diff. regime	3%	−0.173	−2.61
AL_8	Diff. regime	0%	−0.155	−0.69
AL_9	Diff. regime	8%	−0.213	−15.17
AL_10	Diff. regime	0%	−0.183	−11.16
<i>Overall^a</i>	—	49%	+0.002	+0.58

Overall row weights per-target values by per-target pair count, not unweighted mean; unweighted mean over the eight per-target Avg ΔMAE values is +0.0034.

Isolating averaging from ML blending. The headline configuration in Table 9 entangles three design choices: (i) within-source-run ensemble-averaging across the $K=2$ transferred formulas, (ii) the 50/50 ML+formula blend, and (iii) the $\Delta R^2_{\text{vs-ML}} > 0$ source-side filter. Table 10 reports four diagnostic configurations that vary one or more of these choices while holding the rest of the protocol (no retraining, no coefficient re-fitting, zero LLM calls) fixed.

Table 10: Cross-plate transfer ablations. “Same” aggregates targets AL_2–AL_5; “Diff” aggregates AL_7–AL_10. The headline $K=2$ configuration produces 148 source-run $\times$ target pairs per cohort, or 296 individual-formula $\times$ target evaluations per cohort under per-individual-formula scoring. The unfiltered (64 source runs) and below-filter-only (27 source runs) rows scale accordingly. Avg ΔMAE is positive when the transferred formula reduces error relative to ML-only. The cohort/anti-cohort split is preserved across all four configurations.

Configuration	Same reagent (AL_2–AL_5)		Diff. regime (AL_7–AL_10)	
	% improving	Avg ΔMAE	% improving	Avg ΔMAE
<i>Filtered source pool</i>				
Averaged + 50/50 ML blend (headline)	95.3% (141/148)	+0.188	2.7% (4/148)	−0.181
Per-indiv. formula + 50/50 ML blend	78.0% (231/296)	+0.114	11.5% (34/296)	−0.207
Averaged, formula-only (no ML blend)	64.2% (95/148)	+0.073	6.8% (10/148)	−0.146
Per-indiv. formula, no ML blend (joint)	51.7% (153/296)	+0.041	8.8% (26/296)	−0.169
<i>Filter sensitivity (averaged + ML blend)</i>				
Unfiltered all-source (64 runs)	71.1% (182/256)	+0.103	5.1% (13/256)	−0.171
Below-filter only (27 runs,)	38.0% (41/108)	−0.014	8.3% (9/108)	−0.158

What the ablations show. Stripping the within-run ensemble averaging (row 2) reduces within-cohort improvement rate from 95% to 78% and the within-cohort ΔMAE from +0.188 to +0.114; across-cohort improvement remains in the low double digits (11.5%, $\Delta\text{MAE} = -0.207$). Stripping the ML blend instead (row 3) attenuates more strongly — formula-only predictions on bounded $[0, 1]$ targets are noisier when the source-plate residual no longer applies — but again preserves the cohort split (64.2% vs. 6.8%). The joint ablation (row 4) is the cleanest single-formula test: 51.7% within-cohort vs. 8.8% across-cohort improvement, a $\sim 6\times$ asymmetry. This eliminates the possibility that a near-zero or constant formula is driving Table 9 purely through the ML-blend regularizer.

The filter-sensitivity rows show a complementary pattern. Removing the source-side filter (row 5) lowers within-cohort improvement from 95.3% to 71.1% and within-cohort ΔMAE from +0.188 to +0.103, while the across-cohort floor moves only modestly (2.7% $\rightarrow$ 5.1%, $-0.181 \rightarrow -0.171$). The 27 source runs that did not improve on their own training plate (row 6) also do not transfer: 38.0% within-cohort improvement with a slightly negative ΔMAE , close to the across-cohort baseline. This supports our reading of the filter as a noise-reduction step on source-side residual quality rather than a circular validity criterion — below-filter sources behave like noise-fits, and the filter selects sources whose residuals carry transferable signal.

Base-model strength governs transfer reliability. Within-cohort, XGBoost-base source runs transfer more consistently (67/128 improving pairs, avg $\Delta\text{MAE} = +0.015$) than Linear-trusted runs (77/168 improving pairs, avg $\Delta\text{MAE} = -0.007$).¹ A stronger base-model leaves a smaller, more structured residual for MARICL to explain, and the resulting formula is correspondingly tighter and less contaminated by plate-specific variation. Transferability is thus not an accidental property of the LLM but a controllable consequence of residual quality. We additionally provide: (i) a source $\times$ target heatmap of ΔMAE ; (ii) breakdown by source-run base-model and refinement length; and (iii) per-source analysis showing that structurally simpler corrections (smaller K , fewer refinement iterations) are also the most transferable within cohort.

Caveats and isolating ablations. The headline transfer prediction is $\hat{y} = 0.5 \cdot \hat{y}_{\text{ML}} + 0.5 \cdot \text{mean}_m(\hat{y}_m)$, which entangles two effects: (i) within-run ensemble-averaging across the $K=2$ transferred mechanisms acts as a regularizer; (ii) the 50/50 ML blend shrinks predictions toward the ML prior. A constant or near-zero formula could in principle appear to “improve” or “fail” purely through this dynamic. To isolate each effect we report three additional configurations in Appendix E (Table 10): per-individual-formula transfer (no within-run averaging), formula-only transfer (no ML blend), and the joint ablation (no averaging *and* no blend). The cohort/anti-cohort asymmetry survives every condition: the joint ablation, which is the cleanest single-formula test, still yields 51.7% within-cohort improvement vs. 8.8% across-cohort, a $\sim 6\times$ asymmetry that cannot be attributed to either regularizer.

E.1 Implementation

After MARICL training on a source plate, the inferred correction formula is a Python-evaluable string stored in the mechanism file `mechanisms_iter_{T}.txt`. The transfer experiment tests whether this frozen formula, evaluated numerically on a different target plate, still reduces prediction error; without any further LLM calls, retraining, or hyperparameter tuning. We scan all subdirectories of the result folder matching `bio_reg_protein_expression_plate*` and load each run’s `final_results.json`. A run is retained as a valid source if its post-training R^2 exceeds the ML-only baseline on the source plate:

$$\Delta R^2_{\text{vs-ML}} = R^2_{\text{post}} - R^2_{\text{ML-baseline}} > 0.$$

This filters to runs where MARICL genuinely learned something beyond the base model, ensuring that only meaningful formulas enter the transfer pool. In our corpus, 37 of 64 runs pass this filter across two source plates (AL_2 and AL_6); Table 4.

For each (source run, target plate) pair:

1. **Target plate loading.** The target plate CSV is loaded and split into 80/20 train/test using quantile-stratified sampling (mirroring the training protocol of Script 018).

¹These counts and averages are computed under the headline averaged+blend configuration (row 1 of Table 10); the base-model trend reproduces qualitatively under each of the four ablation rows but with smaller absolute magnitudes.

2. **ML mechanism.** The ML model (Linear or XGBoost, as used in the source run) is either (a) transferred directly from the source plate or (b) retrained on the target plate’s train split, depending on the `-ml_source` setting. In all reported results we use `auto` mode, which trains on the source plate when the plate index is detectable from the directory name.
3. **Feature scaling.** All features are scaled to $[0.01, 0.99]$ using `MinMaxScaler010` fit on the train split of whichever plate the ML model was trained on, then applied to the target test set.
4. **Formula evaluation.** The extracted formula string is evaluated via Python’s `eval()` with the test feature matrix injected as local variables. NumPy operations (`np.clip`, `np.exp`, etc.) are available. Outputs are clipped to $[0, 1]$ for stability.
5. **Prediction blending.** Final predictions are a 50/50 blend of the ML mechanism output and the average formula output across all transferred LLM mechanisms:

$$\hat{y} = 0.5 \cdot \hat{y}_{\text{ML}} + 0.5 \cdot \frac{1}{|M_{\text{LLM}}|} \sum_{m \in M_{\text{LLM}}} \hat{y}_m.$$

This blend is a different predictor from Eq. 1; we adopt it for transfer because (i) treating formula outputs as absolute predictions on the bounded $[0, 1]$ scale is more robust than treating them as residual corrections when the source ML residual no longer applies to the target plate, and (ii) $\beta_{\text{transfer}}=0.5$ equally weights the two predictors under transfer uncertainty (Section 2). A pilot residual-based evaluation ($\hat{y} = \hat{y}_{\text{ML}} + \frac{1}{2}(\hat{f}_{\text{LLM}} - \bar{y}_{\text{train}})$ centred at the scaled training mean) gave similar results but is not used in the main sweep.

6. **Baseline.** The ML-only condition uses $\hat{y} = \hat{y}_{\text{ML}}$ with no formula contribution.

F Computational Cost Analysis

Table 11: Computational cost comparison. MARICL incurs one-time training cost but zero inference overhead. Costs estimated with Gemini 2.0 Flash (\$0.10/1M input, \$0.40/1M output tokens), assuming 5K input and 2K output tokens per call (\$0.0013 per call). Batch size $B = 10$ for encoding.

Method	Encoder Calls	Decoder/Refine Calls	Total LLM Calls	Training Cost	1K Queries Cost
Direct LLM (ICL)	–	1 per query	1K	\$0.00	\$1.30
MARICL ($K=1, T=5, \mathcal{D}_{\text{high-res}} =30$)	3	11	14	\$0.018	\$0.018
MARICL ($K=2, T=5, \mathcal{D}_{\text{high-res}} =50$)	10	22	32	\$0.042	\$0.042
MARICL ($K=2, T=10, \mathcal{D}_{\text{high-res}} =100$)	20	42	62	\$0.081	\$0.081
MARICL ($K=2, T=10, \mathcal{D}_{\text{high-res}} =200$)	40	42	82	\$0.107	\$0.107

Table 11 compares computational costs across methods, accounting for MARICL’s batched encoding strategy. Direct LLM inference requires one LLM call per prediction, costing \$0.0013 per query and scaling to \$1.30 for 1K queries.

MARICL’s LLM calls during training consist of two components based on Algorithm 1:

1. **Encoder calls:** For each correction k , encoding requires $\lceil |\mathcal{D}_{\text{high-res}}|/B \rceil$ LLM calls when batched encoding is used (Eq. 7). With K corrections, this totals $K \cdot \lceil |\mathcal{D}_{\text{high-res}}|/B \rceil$ encoder calls.
2. **Decoder and refinement calls:** Initial decoding requires K calls (Eq. 8). Each refinement iteration requires one critique generation (Eq. 11) and one correction refinement (Eq. 13) per correction, totaling $2KT$ calls over T iterations. Combined: $K(1 + 2T)$ decoder/refinement calls.

The total number of LLM calls is therefore:

$$N_{\text{calls}} = K \cdot \left\lceil \frac{|\mathcal{D}_{\text{high-res}}|}{B} \right\rceil + K(1 + 2T) \quad (19)$$

For a typical configuration with $K=2$ corrections, $T=10$ iterations, $|\mathcal{D}_{\text{high-res}}|=100$ samples, and batch size $B=10$, this yields $2 \cdot 10 + 2 \cdot 21 = 62$ total LLM calls at \$0.081 training cost. Crucially, MARICL

Table 12: Performance across LLM backbones on Cell-Free Protein (XGBoost base-model) and Zoo (Logistic base-model). $K = 1, T = 5$. “Corr” = correction only; “Full” = combined MARICL.

LLM Backbone	Protein (R^2)		Zoo (Acc)	
	Corr	Full	Corr	Full
Gemini-2.5-Flash	0.52	0.74	0.81	0.981
GPT-4o	0.54	0.73	0.83	0.987
Gemini-2.0-Flash	0.50	0.72	0.76	0.975
GPT-4o-mini	0.46	0.68	0.71	0.95
Llama-3-8B	0.38	0.62	0.62	0.86

inference requires *zero* LLM calls; corrections compile to executable Python formulas evaluated directly on input features. This achieves over 93% cost reduction at 1K queries (\$0.081 vs. \$1.30) and cost parity with traditional ML at inference time, with savings increasing linearly with query volume. Note that encoder calls scale with $K \cdot \lceil |\mathcal{D}_{\text{high-res}}|/B \rceil$, while decoder/refinement calls scale with $K \cdot T$. For datasets requiring larger high-residual subsets, encoder calls may dominate; for complex corrections requiring many refinement iterations, decoder calls dominate.

Total experiment cost. All experiments reported in this paper were executed on a MacBook Pro with M1 Max chip (32 GB unified memory) for base-model training and local Python execution, with LLM calls routed to cloud endpoints (Gemini 2.0 Flash via Google AI Studio, GPT-4o / o1-preview via OpenAI API). Aggregating across all reported runs—9 benchmark datasets $\times$ 5 seeds per dataset for headline tables, plus ablation sweeps ($K \in \{1, 2\}$, $T \in \{0, 3, 5, 10\}$, Top-K $\in \{30, 50, 100\}$, scaling modes, LLM backbone comparison, domain-context ablation, anonymized-features ablation, joint prior-stripping ablation), cross-plate transfer experiments (37 source runs $\times$ 8 target plates), and synthetic benchmark recovery—the full experimental budget is estimated at $\leq \$250$ using the per-call cost in Table 11. Exact wall-clock call counts will be released with code. Compute-infrastructure requirements are minimal: MARICL trains on a laptop and requires no GPU, cluster, or high-memory node. Per-dataset training completes in under 5 minutes of wall-clock time (excluding LLM API latency).

G LLM Backbone Analysis

Table 12 examines how LLM backbone choice affects MARICL performance. We evaluate frontier models (GPT-4o, Gemini-2.5-Flash), efficient alternatives (GPT-4o-mini, Gemini-2.0-Flash), and open-source options (Llama-3-8B). GPT-4o produces corrections with the highest standalone R^2 (0.54) and accuracy (0.83), suggesting stronger reasoning capabilities translate to better hypothesis generation. Gemini-2.0-Flash achieves 99% of GPT-4o’s full performance ($R^2 = 0.72$ vs. 0.73) at approximately 1/20th the cost (\$0.006 vs. \$0.14 training). Llama-3-8B achieves $R^2 = 0.62$ (51% improvement over Linear Regression of 0.41), demonstrating that moderately sized open models can generate useful corrections and enable local deployment of MARICL.

H Additional Correction Analysis

Table 13 presents corrections inferred for remaining datasets, including cases where MARICL shows limited improvement or reveals domain complexity limitations.

The enzyme activity correction demonstrates that MARICL can recover established biochemical principles through data-driven refinement. The Michaelis-Menten saturation term for sequence length follows the canonical form $\frac{V_{\text{max}} \cdot S}{K_m + S}$, where enzyme activity increases with substrate concentration but saturates at high levels [Cornish-Bowden, 2012]. The Gaussian optimum term $\exp(-(x - 5)^2/8)$ models the observation that aromatic residue content has an optimal range for protein stability; too few aromatic residues reduce hydrophobic core stability, while too many can cause aggregation [Pace et al., 1996]. The polar residue inhibition term captures competitive dynamics where excess polar residues at the active site can disrupt substrate binding. XGBoost+MARICL achieves $R^2 = 0.5132$, representing a 28.6% improvement over XGBoost alone ($R^2 = 0.3992$), though the moderate absolute

Table 13: Corrections inferred for enzyme activity, diabetes progression, social classification, and income prediction. The “Correction Only” column shows performance using only the learned correction without the base-model, while “MARICL” shows the full combined performance.

Dataset	Learned Correction Formula	Domain Interpretation & Literature Support	Correction Only	MARICL
Enzyme Activity	$\hat{y} = \text{clip}\left(\frac{6.0 \cdot \text{seq_length}}{0.3 + \text{seq_length}} \cdot \frac{\text{seq_gc_content}}{1 + 0.5 \cdot \frac{(\text{seq_aromatic} - 5.0)^2}{8.0}} \cdot \frac{1}{1 + 0.3 \cdot \text{seq_polar}}, 0, 10\right)$	Michaelis-Menten saturation: The term $\frac{\text{seq_length}}{0.3 + \text{seq_length}}$ captures substrate binding kinetics where activity saturates at high sequence lengths [Cornish-Bowden, 2012]. Gaussian aromatic optimum: The $\exp(-(x - 5)^2/8)$ term models optimal aromatic residue content, consistent with hydrophobic core requirements for protein stability [Pace et al., 1996]. Polar residue inhibition: The $\frac{1}{1 + 0.3 \cdot \text{seq_polar}}$ term captures competitive inhibition dynamics from excess polar residues disrupting active site geometry. We note that while Michaelis-Menten kinetics are within LLM pretraining knowledge, the specific parameterization and the combination with aromatic Gaussian and polar inhibition terms were refined iteratively through residual-driven TextGrad (see Section 4.3).	0.421	0.513 R^2
Diabetes	$\hat{y} = \text{clip}\left((0.591 \cdot s5 + 0.509 \cdot s1 + 0.347 \cdot \text{BMI} + 0.312 \cdot s2 + 0.200 \cdot s5 \times s1 + 0.200 \cdot s5 \times \text{BMI} + \dots)/12, 0, 1\right)$	Triglyceride dominance: The highest weight on $s5$ (0.591) aligns with clinical evidence that log-transformed triglycerides are among the strongest predictors of diabetes progression [DeFronzo et al., 2015]. Metabolic interactions: The $s5 \times s1$ and $s5 \times \text{BMI}$ interaction terms capture compounding metabolic dysregulation, consistent with the established relationship between dyslipidemia, obesity, and insulin resistance [Kahn et al., 2006]. Cholesterol contribution: The $s1$ weight (0.509) reflects total cholesterol’s role in cardiovascular comorbidity prediction.	0.482	0.590 R^2
Social Groups	$\text{score}_{\text{Grades}} = 1.2 \cdot \frac{\text{Grades}}{0.5 + \text{Grades}} + 0.6 \cdot \text{Race} - (1 - \text{Gender}) \cdot 0.5 \cdot \text{score}_{\text{Sports}} = 1.6 \cdot \text{Sports} + (1 - \text{Gender}) \cdot 1.3 - 0.4 \cdot \text{Race}$	Direct feature mapping: Unlike biochemical domains, the correction relies on surface-level feature weighting (Sports $\rightarrow$ Sports group) rather than inferring latent social dynamics. Limited depth: The saturation term $\frac{\text{Grades}}{0.5 + \text{Grades}}$ attempts nonlinearity but demographic features are weak proxies for complex social processes governed by unobserved confounders [McPherson et al., 2001]. This is a representative failure mode: MARICL correctly provides minimal correction via the $p_{\min}$ threshold when the domain lacks correctable structure.	0.391	0.540 Acc
Adult Income	$\text{score}_{>50K} = 0.344 \cdot \frac{\text{capital_gain}}{0.4 + \text{capital_gain}} + 0.309 \cdot \frac{\text{relationship}}{0.4 + \text{relationship}} + 0.236 \cdot \frac{\text{capital_gain} \times \text{education_num}}{0.3 + \dots}$	Capital gains saturation: The $\frac{\text{capital_gain}}{0.4 + \text{capital_gain}}$ term captures diminishing marginal returns on investment income, consistent with economic utility theory [Kahneman and Tversky, 1979]. Education-wealth interaction: The $\text{capital_gain} \times \text{education_num}$ term reflects compounding effects where higher education amplifies returns on capital investments. Relationship status: The 0.309 weight captures household economic structure effects on income classification.	0.712	0.832 Acc

performance suggests sequence-activity relationships involve complexity beyond simple correction formulas.

The diabetes correction identifies clinically validated predictors with appropriate weightings. The dominance of $s5$ (log-transformed triglycerides, weight 0.591) aligns with extensive clinical literature showing triglycerides as one of the strongest metabolic predictors of diabetes progression [DeFronzo et al., 2015]. The interaction terms $s5 \times s1$ (triglycerides $\times$ cholesterol) and $s5 \times \text{BMI}$ capture the compounding effects of metabolic syndrome, where dyslipidemia and obesity synergistically accelerate insulin resistance [Kahn et al., 2006]. Linear+MARICL achieves $R^2 = 0.5900$, a $\Delta R^2 = 0.136$ improvement over Linear Regression alone ($R^2 = 0.4543$), consistent with clinical understanding of diabetes as a multifactorial metabolic disorder.

In contrast, the social and income corrections reveal important domain-dependent behaviors. The High School correction relies on direct feature weighting rather than inferring hypothetical social dynamics; the Sports feature directly predicts Sports group membership without uncovering latent social structures. The saturation term $\frac{\text{Grades}}{0.5 + \text{Grades}}$ attempts to capture nonlinear grade effects, but

demographic features are fundamentally weak proxies for complex social processes governed by unobserved confounders such as peer influence, family background, and school culture [McPherson et al., 2001]. XGBoost+MARICL achieves accuracy of 0.5400, only a modest improvement over XGBoost alone (0.5170), confirming that domains governed by latent social dynamics may not benefit substantially from MARICL’s correction approach. We highlight this as a representative failure mode rather than obscuring it.

The Adult Income results demonstrate stronger correction grounding and consistent performance gains. The capital gains saturation term $\frac{\text{capital.gain}}{0.4+\text{capital.gain}}$ follows diminishing marginal utility principles from behavioral economics [Kahneman and Tversky, 1979], where additional investment income provides decreasing relative benefit at higher levels. The education-wealth interaction term captures the well-documented phenomenon that higher education amplifies returns on capital investments through financial literacy and investment opportunities. XGBoost+MARICL achieves accuracy of 0.832 and macro F1 of 0.800, outperforming XGBoost alone (accuracy 0.813, macro F1 0.692). The macro F1 improvement (+15.6% relative; +0.108 absolute) reflects MARICL’s ability to address class imbalance through targeted corrections. Notably, MARICL helps both weak (Logistic: macro F1 improves from 0.679 to 0.740) and strong (XGBoost: macro F1 improves from 0.692 to 0.800) base-models on this dataset, with the largest lift coming from minority-F1 recovery rather than majority precision. We note that the lift on XGBoost reflects its initially low minority-class recall (Minority F1 = 0.498 despite Accuracy = 0.813); MARICL’s residual-driven corrections concentrate on these minority-class errors, raising minority F1 to 0.720.

I Anonymized Features on Real Data

The domain-context ablation in Section 4.3 keeps feature names intact, which themselves carry semantic content (e.g., “NAD”, “BMI”, “capital.gain”). To bound the contribution of feature-name priors, we re-run MARICL on Cell-Free Protein and Diabetes with all feature names replaced by opaque identifiers (`feat_0`, ..., `feat_d`) before any prompt construction. Numerical values, residuals, and the iterative refinement loop are otherwise identical.

Table 14: Anonymized-feature ablation. “Anonym.” replaces feature names with opaque identifiers. The data-driven fraction $G_{\text{anon}}/G_{\text{full}}$ estimates how much of MARICL’s gain survives when feature semantics are withheld.

Dataset	Linear	MARICL (full)	MARICL (anonym.)	$G_{\text{anon}}/G_{\text{full}}$
Cell-Free Protein	0.412	0.648	0.571	0.67
Diabetes	0.454	0.590	0.548	0.69

On Cell-Free Protein, roughly two-thirds of the gain ($G_{\text{anon}} = 0.159$ of $G_{\text{full}} = 0.236$) is recovered without feature semantics, indicating that the residual-driven loop captures most of the structure from data alone; the remaining third reflects the contribution of biochemistry priors triggered by feature names like “NAD” and “spermidine”. On Diabetes, the data-driven fraction is similar (0.69), consistent with `s1`–`s6` already being partially anonymized in the source data. We report this transparently rather than claiming MARICL is fully prior-free on real benchmarks.

J Complete Experimental Results

Table 15 presents the complete regression results across all three datasets using Gemini-flash-2.0 LLM backbone, and Table 16 presents the complete classification results.

J.1 Decomposing Structural and Numerical Contributions

MARICL’s correction f_k is a symbolic expression with LLM-generated coefficients. Two distinct contributions could explain its performance: the inferred *structure* (which features, which interactions, which nonlinearities) and the *numerical fit* (the coefficient values within that structure). We isolate each by refitting coefficients via ordinary least squares on the training set while holding the structure fixed, and conversely by randomizing coefficients within the inferred structure.

Table 15: Complete regression results across all datasets. Best results are shown in **bold**, second best are underlined. $\uparrow$ indicates higher is better, $\downarrow$ indicates lower is better.

Method	Cell-Free Protein		Enzyme Activity		Diabetes Progression	
	$R^2 \uparrow$	MAE $\downarrow$	$R^2 \uparrow$	MAE $\downarrow$	$R^2 \uparrow$	MAE $\downarrow$
<i>Traditional ML Baselines</i>						
Linear Regression	0.4115 $\pm$ 0.0200	0.1413 $\pm$ 0.0050	0.2373 $\pm$ 0.0100	0.1931 $\pm$ 0.0050	0.4543 $\pm$ 0.0200	0.1586 $\pm$ 0.0050
XGBoost	0.5787 $\pm$ 0.0250	0.1090 $\pm$ 0.0050	0.3992 $\pm$ 0.0200	0.1655 $\pm$ 0.0050	0.3080 $\pm$ 0.0150	0.1647 $\pm$ 0.0050
<i>Interpretable ML Models</i>						
EBM	0.5946 $\pm$ 0.0250	0.1169 $\pm$ 0.0050	0.4160 $\pm$ 0.0200	0.1580 $\pm$ 0.0050	0.5389 $\pm$ 0.0250	0.1487 $\pm$ 0.0050
Symbolic Regression	0.4500 $\pm$ 0.0200	0.1350 $\pm$ 0.0050	0.2500 $\pm$ 0.0100	0.1850 $\pm$ 0.0050	0.4800 $\pm$ 0.0200	0.1520 $\pm$ 0.0050
<i>Neural Tabular Models</i>						
TabPFN	<u>0.6782</u> $\pm$ 0.0300	<u>0.1007</u> $\pm$ 0.0050	0.4128 $\pm$ 0.0100	0.1683 $\pm$ 0.0050	0.6170 $\pm$ 0.0300	0.1352 $\pm$ 0.0050
<i>LLM-Based Methods</i>						
LLM-LEX	0.3308 $\pm$ 0.0500	0.2048 $\pm$ 0.0200	0.2550 $\pm$ 0.0600	0.2553 $\pm$ 0.0200	0.3457 $\pm$ 0.0300	0.2416 $\pm$ 0.0100
LLM-ICL	0.3500 $\pm$ 0.0200	0.1500 $\pm$ 0.0050	0.2200 $\pm$ 0.0100	0.2000 $\pm$ 0.0050	0.4000 $\pm$ 0.0200	0.1700 $\pm$ 0.0050
<i>MARICL (Ours)</i>						
Linear + MARICL	0.6475 $\pm$ 0.0300	0.0991 $\pm$ 0.0050	<u>0.4812</u> $\pm$ 0.0250	<u>0.1493</u> $\pm$ 0.0050	<u>0.5900</u> $\pm$ 0.0300	<u>0.1401</u> $\pm$ 0.0050
XGBoost + MARICL	0.7231 $\pm$ 0.0300	0.0937 $\pm$ 0.0050	0.5132 $\pm$ 0.0100	0.1436 $\pm$ 0.0050	0.5430 $\pm$ 0.0250	0.1520 $\pm$ 0.0050

Table 16: Complete classification results across all datasets. Best results are shown in **bold**, second best are underlined. $\uparrow$ indicates higher is better.

Method	Zoo (7 classes)		High School (3 classes)		Adult Income (2 classes)	
	Accuracy $\uparrow$	F1 $\uparrow$	Accuracy $\uparrow$	F1 $\uparrow$	Accuracy $\uparrow$	Macro F1 $\uparrow$
<i>Traditional ML Baselines</i>						
Logistic Regression	0.8570 $\pm$ 0.0300	0.8330 $\pm$ 0.0300	0.4670 $\pm$ 0.0200	0.4080 $\pm$ 0.0200	0.7380 $\pm$ 0.0200	0.6790 $\pm$ 0.0200
XGBoost	0.9050 $\pm$ 0.0300	0.8860 $\pm$ 0.0300	0.5170 $\pm$ 0.0200	0.3520 $\pm$ 0.0200	0.8130 $\pm$ 0.0250	0.6920 $\pm$ 0.0250
<i>Interpretable ML Models</i>						
EBM	0.9048 $\pm$ 0.0300	0.8836 $\pm$ 0.0300	0.5333 $\pm$ 0.0200	0.4984 $\pm$ 0.0200	0.8125 $\pm$ 0.0250	0.7350 $\pm$ 0.0250
Symbolic Regression	0.8700 $\pm$ 0.0200	0.8500 $\pm$ 0.0200	0.4800 $\pm$ 0.0200	0.4200 $\pm$ 0.0200	0.7500 $\pm$ 0.0200	0.6500 $\pm$ 0.0200
<i>Neural Tabular Models</i>						
TabPFN	<u>0.9524</u> $\pm$ 0.0250	<u>0.9365</u> $\pm$ 0.0250	0.5500 $\pm$ 0.0200	0.4732 $\pm$ 0.0200	0.8525 $\pm$ 0.0300	0.8180 $\pm$ 0.0300
<i>LLM-Based Methods</i>						
LLM-LEX	0.2000 $\pm$ 0.0100	0.2400 $\pm$ 0.0100	0.5000 $\pm$ 0.0200	0.4485 $\pm$ 0.0200	0.6000 $\pm$ 0.0200	0.5128 $\pm$ 0.0200
LLM-ICL	0.8000 $\pm$ 0.0200	0.7800 $\pm$ 0.0200	0.4500 $\pm$ 0.0200	0.3800 $\pm$ 0.0200	0.7000 $\pm$ 0.0200	0.6000 $\pm$ 0.0200
<i>MARICL (Ours)</i>						
Logistic + MARICL	0.9750 $\pm$ 0.0200	0.9700 $\pm$ 0.0200	0.5210 $\pm$ 0.0200	0.4830 $\pm$ 0.0200	0.8130 $\pm$ 0.0250	0.7400 $\pm$ 0.0250
XGBoost + MARICL	0.9520 $\pm$ 0.0250	0.9490 $\pm$ 0.0250	<u>0.5400</u> $\pm$ 0.0200	0.5100 $\pm$ 0.0200	<u>0.8324</u> $\pm$ 0.0250	<u>0.8000</u> $\pm$ 0.0250

The decomposition shows that the inferred structure is the dominant contribution: random coefficients within MARICL’s structure already recover 36–44% of the gain over the baseline. LSQ refitting adds a further 0.012–0.048 R^2 , indicating that LLM-generated coefficients are sound but not numerically optimal within their own structure. Practitioners who require the last fraction of accuracy can post-hoc refit; those who prefer fully LLM-generated formulas (e.g., for faithful representation of the agent’s reasoning) lose only modest performance. We treat structural inference as MARICL’s primary contribution and numerical fitting as a complementary post-processing step.

K Statistical Significance

For each MARICL variant we compute paired Wilcoxon signed-rank tests against the strongest non-MARICL baseline on each dataset, using per-(seed, fold) paired metric values across 5 seeds $\times$ 5 cross-validation folds ($n=25$ paired observations per dataset; the 5-fold CV protocol matches Table 22). At $n=25$ the smallest possible exact two-sided p -value is $\approx 6 \times 10^{-8}$, so the raw p -values reported below are well within the attainable range of the test. We apply Benjamini–Hochberg correction at FDR 0.05 across the 9 dataset-level comparisons, computing $q_{(i)} = \min_{j \geq i} \frac{m}{j} p_{(j)}$ with $m=9$. Table 18 reports raw and corrected p -values. Results are reported for the better-performing MARICL variant on each dataset (Linear+MARICL or XGBoost+MARICL).

Reading guide. Four of nine MARICL improvements survive Benjamini–Hochberg correction at FDR 0.05 (Enzyme Activity, Synthetic, California Housing, Bike Sharing). Two further datasets are borderline at the 0.05 threshold (Cell-Free Protein $q=0.056$; Zoo $q=0.059$); their raw p -values would clear FDR control at any threshold ≥ 0.06 . The three remaining comparisons (Diabetes, High School, Adult Income) correspond to datasets where TabPFN is competitive with or stronger than

Table 17: Structure-vs-coefficient decomposition (Linear, R^2). “LSQ-refit” replaces LLM coefficients with least-squares values; “Random-coef” samples coefficients uniformly in $[-1, 1]$ within MARICL’s structure (mean over 20 random draws).

Configuration	Cell-Free	Diabetes	Enzyme
Linear Base-model	0.412	0.454	0.237
MARICL (full)	0.648	0.590	0.481
MARICL + LSQ-refit	0.661	0.602	0.529
MARICL + Random-coef	0.498	0.512	0.341

Table 18: Paired Wilcoxon signed-rank tests, MARICL (best variant) vs. strongest non-MARICL baseline. Δ denotes the metric improvement (positive favors MARICL). p_{BH} is the Benjamini–Hochberg-corrected p -value at FDR 0.05. Bold Δ indicates corrected $p < 0.05$.

Dataset	Strongest baseline	Metric	Δ	p raw	p_{BH}
Cell-Free Protein	TabPFN	R^2	+0.045	0.031	0.056
Enzyme Activity	EBM	R^2	+0.097	0.008	0.045
Diabetes	TabPFN	R^2	−0.027	0.094	0.106
Cal. Housing	XGBoost	R^2	+0.029	0.016	0.045
Bike Sharing	XGBoost	R^2	+0.023	0.020	0.045
Zoo	TabPFN	Accuracy	+0.023	0.039	0.059
High School	TabPFN	Accuracy	−0.010	0.156	0.156
Adult Income	TabPFN	Accuracy	−0.020	0.078	0.100
Synthetic	PySR-on-resid.	R^2	+0.038	0.012	0.045

MARICL on raw accuracy. On these datasets, MARICL’s contribution is the interpretable closed-form correction formula rather than a statistically significant improvement in predictive accuracy. We highlight this honestly rather than reporting only the headline metric: MARICL is most clearly advantageous on domains with correctable mechanistic structure (Enzyme Activity, Synthetic, the TabArena benchmarks, and – borderline – Cell-Free Protein and Zoo), and ties or slightly trails the strongest tabular foundation model on benchmarks where the predictive signal is dominated by features the base-model already exploits.

L Joint Prior-Stripping Ablation

The single-channel ablations in Appendices C withhold one source of LLM prior knowledge, but cannot bound the joint contribution: feature names alone may cue domain priors that survive context removal, and a strong backbone may compensate for either. We therefore report the combined ablation, which represents the most prior-stripped configuration achievable without altering the algorithm: feature names replaced with opaque identifiers (`feat_0, . . . , feat_d`), no domain context in any prompt, and Llama-3-8B as the LLM backbone in place of frontier models.

Approximately half of MARICL’s end-to-end gain survives joint prior-stripping ($G_{\text{joint}}/G_{\text{full}} \approx 0.49\text{--}0.50$ across three datasets). The surviving half characterizes the residual-driven loop in isolation: a refinement procedure that operates on data alone and produces gains comparable to

Table 19: Joint prior-stripping ablation (Linear base-model, $K = 2$, $T = 10$). $G_{\text{joint}}/G_{\text{full}}$ is the fraction of MARICL’s gain that survives all three ablations simultaneously. PySR-on-residuals included as the strongest fully prior-free baseline.

Dataset	Linear	PySR-res	Joint-strip	Full	$G_{\text{joint}}/G_{\text{full}}$
Cell-Free	0.412	0.471	0.527	0.648	0.49
Diabetes	0.454	0.512	0.521	0.590	0.49
Enzyme	0.237	0.318	0.358	0.481	0.50

or exceeding PySR-on-residuals (Table 19), the strongest fully prior-free symbolic baseline. The remaining half characterizes what MARICL extracts when its priors align with the domain: a deployment benefit available wherever the LLM’s pretraining is informative for the task at hand.

We frame these as complementary rather than competing contributions. The data-driven half is what makes MARICL applicable in prior-poor settings (anonymized features, novel domains, on-prem

Table 20: TabArena benchmark results. MARICL provides consistent improvements across both base models on larger-scale datasets with less explicit feature semantics.

Task (N / d)	Linear	Linear+MARICL	XGBoost	XGB+MARICL
California (20,640 / 8)	0.576	0.648 (+0.072)	0.832	0.861 (+0.029)
Bike Sharing (17,389 / 12)	0.391	0.493 (+0.102)	0.894	0.917 (+0.023)

open-source backbones) where prior-dependent methods would not function. The prior-aligned half is what makes MARICL more useful than a symbolic regressor on real scientific benchmarks where domain knowledge is freely available in pretraining corpora and not exploiting it would be wasteful. The inference argument is established separately through the synthetic benchmark (§4.2), where priors are absent by construction, and the cross-plate transfer experiment (§4.5), where mechanism is testable through cohort-aligned generalization; the present ablation does not aim to establish discovery but to make explicit what each channel contributes.

M Validation Protocol and Generalization Safeguards

Because textual gradient optimization uses training performance to generate critiques and select the best iteration, we formally clarify the role of each data split and provide both structural and empirical safeguards against overfitting.

Strict data partitioning. All datasets are split into training, validation, and test sets. Residual computation and high-residual selection use training data only (Eq. 4). The held-out test set is never accessed until final evaluation. No test-set information influences any design decision, hyperparameter choice, or correction refinement.

Capacity bound of the hypothesis space. Corrections are constrained to 3–8 component symbolic formulas built from a restricted operation set (addition, multiplication, clipping, sigmoid, rational saturation $x/(K + x)$, Gaussian $\exp(-(x - \mu)^2/\sigma^2)$; see Appendix N.3). With d features and at most 8 components, the effective number of structural degrees of freedom is on the order of $\mathcal{O}(d \cdot 8)$, far below the validation set sizes (88–6,512 samples). This structural constraint makes memorization of examples infeasible, independent of how many refinement iterations are run.

Lossy critique channel. Unlike numerical optimization where gradients transmit per-example information, textual critiques are lossy natural language summaries (e.g., “overcorrects for high-value samples”). The LLM receives aggregate error pattern descriptions, not raw examples or numerical gradients. This information bottleneck further limits the effective bandwidth available for overfitting.

Analogy to standard model selection. Neural architecture search, early stopping, AutoML, and pruning is similar to gradient-based training. The key distinction from gradient-based overfitting is that each refinement step produces a discrete, structurally constrained candidate rather than incrementally adjusting continuous parameters toward the validation surface.

M.1 Extended Benchmarks: TabArena

To evaluate MARICL at larger scale and on datasets with less transparent feature semantics, we include two widely used TabArena regression benchmarks (Table 20).

On California Housing, MARICL inferred a location-density interaction (latitude $\times$ longitude $\times$ median_income saturation) capturing known geographic price gradients, interpretable to urban planners. On Bike Sharing, MARICL identified temperature-humidity interaction terms with hour-of-day modulation. These datasets are 100–200 $\times$ larger than our smallest benchmarks and have less semantically transparent features than Zoo or Enzyme, confirming that MARICL scales beyond small, domain-rich settings.

Table 21: Correction stability across 5 random seeds. High School shows least stability, consistent with weak domain structure.

Dataset	Correction Form	Recovered ($N/5$)	Coeff. CV (%)
Diabetes	$s5 \times \text{BMI} + s5 \times s1$ interaction	4/5	11.7
Zoo	hair $\times$ milk $\times$ (1-eggs)	5/5	4.2
Enzyme	Michaelis-Menten + aromatic Gaussian	4/5	13.8
Adult Income	capital.gain saturation + edu. interaction	5/5	8.1
High School	Sports $\rightarrow$ Sports direct mapping	3/5	22.4

M.2 Stability of Learned Corrections

A critical question is whether MARICL’s corrections are stable across independent runs or are artifacts of LLM stochasticity. We assess stability across three axes using 5 seeds.

Random seeds (Table 21): Dominant correction forms are recovered in 4–5 of 5 independent runs on all datasets except High School (3/5), which has the weakest mechanistic structure. Coefficient variation is below 14% for well-structured domains. **Data splits:** The top-ranked correction’s structural form remained consistent in 4+ of 5 folds on Zoo, Adult Income, and Diabetes. **Prompt paraphrasing:** Three semantically equivalent prompt variants (original, shorter, restructured) produce consistent correction categories with performance variance $< 0.02 R^2$. Low cross-seed consistency on High School (CV = 22.4%) is itself a diagnostic signal for insufficient domain structure, not a failure of the method.

M.3 Failure Modes

We consolidate failure modes to characterize when MARICL provides limited benefit:

Base model already captures nonlinear structure. Adding LLM-generated corrections to a stronger base yields strictly smaller within-model gains: $+0.236 \Delta R^2$ for Linear vs. $+0.144 \Delta R^2$ for XGBoost on Cell-Free Protein, and $+0.107$ vs. $+0.033 \Delta R^2$ when going from $K=1$ to $K=2$ corrections under the headline ablation configuration (Table 5). When the base already captures the dominant nonlinearity, the residual signal available for correction shrinks and the relative cost of any LLM-induced noise grows — the limiting case being a strong base with no exploitable residual.

Unobserved confounders. High School gains only $+0.023$ Acc; demographic features are weak proxies for social dynamics governed by peer influence and family background. MARICL correctly identifies this limitation via the $p_{\min}$ threshold rather than hallucinating spurious corrections.

High dimensionality with noise features. On synthetic data with 100 features (10 relevant, 90 noise), performance drops from $R^2 = 0.71$ to 0.68, demonstrating that irrelevant features introduce noise into hypothesis generation.

LLM stochasticity on weakly structured domains. Correction recovery varies directly with domain quality: Zoo 5/5 runs, Adult Income 5/5, but High School only 3/5 (Table 21). Low cross-seed consistency is itself a diagnostic signal for insufficient domain structure.

N Extended Technical Details

This appendix provides additional details on the MARICL framework, including implementation specifics, experimental protocols, and extended analyses that complement the main text.

N.1 Training Protocol and Generalization

The iterative refinement process in MARICL involves generating textual critiques and selecting corrections based on performance. A key design principle is that textual gradients operate fundamentally differently from numerical gradients: they are natural language descriptions of error patterns that must generalize through the LLM’s reasoning capabilities, rather than continuous parameters susceptible to overfitting through gradient descent. Appendix M provides a formal treatment of the validation protocol and generalization safeguards; here we provide additional empirical detail.

Strict Data Partitioning. We enforce complete separation between data splits: the training set provides residuals for correction learning and guides critique generation and correction selection, and the test set remains untouched until final evaluation. No information from the test set influences any design decision, hyperparameter choice, or correction refinement.

Limited Refinement Cycles. We cap refinement at $T = 10$ iterations across all datasets, selecting the best-performing iteration via early stopping. This fixed budget prevents indefinite optimization against the training set. In practice, corrections achieve optimal performance at iteration $t \in [2, 5]$, with later iterations showing diminishing returns or slight degradation.

Correction Complexity Constraints. Generated formulas are constrained to interpretable forms including products, ratios, and saturation terms with typically 3–8 components. This implicit regularization limits the hypothesis space and prevents memorization of data examples. The decoder prompt explicitly requests “simple, interpretable formulas using domain-meaningful transformations.”

To verify these design choices yield robust generalization, we conducted 5-fold cross-validation on three representative datasets. Table 22 demonstrates that test performance closely tracks validation performance, with mean absolute differences below 0.02 across all metrics.

Table 22: Cross-validation results demonstrating validation-test consistency (mean $\pm$ std across 5 folds).

Dataset	Val R^2 / Acc	Test R^2 / Acc	$ \Delta $
Cell-Free Protein	0.718 ± 0.03	0.705 ± 0.04	0.013
Diabetes	0.584 ± 0.02	0.571 ± 0.03	0.013
Adult Income	0.836 ± 0.02	0.824 ± 0.02	0.012

N.2 Hyperparameter Configuration and Sensitivity

Table 23 summarizes the complete hyperparameter configuration used throughout our experiments. Parameters fall into two categories: those selected via grid search on validation data (K , κ , β for *classification* only) and those fixed based on preliminary analysis ($p_{\min}$, γ , B , regression τ).

Table 23: Hyperparameter settings with selection methodology and sensitivity characteristics.

Parameter	Value	Selection	Sensitivity
K (corrections)	2	Grid $\{1, 2, 3, 4\}$	Low ($\pm 5\%$ across range)
κ (residual fraction)	0.3	Grid $\{0.2, 0.3, 0.4, 0.5\}$	Medium ($\pm 8\%$ across range)
T (iterations)	10	Early stopping	Low (optimal at $t \in [2, 7]$)
B (batch size)	10	Context window	Low
$p_{\min}$ (threshold)	0.1	Fixed	Low (filters $< 5\%$ corrections)
β (class. blend)	0.3–0.7	Val. grid $\{0.3, 0.5, 0.7\}$	Medium (dataset dependent)
τ (reg. p_k scale)	$0.2 \times$ target range	Fixed ($\tau=0.2$, $y \in [0, 1]$)	Low
τ_k (softmax temp.)	0.5–3.0	Val. grid; min. ECE	Medium
γ (confidence scale)	2.0	Fixed	Low
τ_{fail} (failure threshold)	0.5	Fixed	Low)

Train vs. validation roles. The global scores p_k in Eq. 17 use MAE_k or macro- F_1 evaluated on the **training** split after m_k^* is selected (same split as Eq. 9; Algorithm 1); these scores are then frozen for inference. Hyperparameters that appear in probability outputs or the classification ensemble — β in Eq. 2, τ_k in Eq. 14, and the headline choices K , κ — are selected by **validation**-set search (ECE for τ_k , validation loss for β). The test set is held out from all of these decisions.

Classification β vs. regression transfer β_{transfer} . Eq. 2 requires $\beta \in [0, 1]$; our search uses $\{0.3, 0.5, 0.7\}$ (*not* $\beta=3$). A value of 3 can arise only as a softmax temperature candidate τ_k in the calibration grid, never as a mixing weight. Cross-plate *regression* transfer instead uses a fixed $\beta_{\text{transfer}} = \frac{1}{2}$ (Section 2), independent of the classification grid; Table 10 reports sensitivity to dropping this blend.

Joint Sensitivity Analysis. Figure 3 presents joint sensitivity results for K and κ on the Cell-Free Protein dataset. Performance is robust across a wide range: $K \in [1, 3]$ and $\kappa \in [0.2, 0.4]$ all achieve $R^2 > 0.70$. The combination $K = 2, \kappa = 0.3$ offers a good balance between correction diversity and focus on systematic errors.

	$\kappa = 0.2$	$\kappa = 0.3$	$\kappa = 0.4$	$\kappa = 0.5$
$K = 1$	0.698	0.705	0.701	0.688
$K = 2$	0.712	0.723	0.718	0.704
$K = 3$	0.708	0.716	0.711	0.695
$K = 4$	0.695	0.702	0.698	0.682

Figure 3: Joint sensitivity of K (corrections) and κ (residual fraction) on Cell-Free Protein (R^2). Performance is stable across $K \in [1, 3]$ and $\kappa \in [0.2, 0.4]$.

Practitioner Guidance. Based on our experiments, we recommend: (1) Start with $K = 2$ corrections, increasing only if performance plateaus; (2) Set $\kappa = 0.3$ as default, adjusting downward for noisy data or upward for systematic model failures; (3) Use early stopping with $T = 10$ maximum iterations; (4) For classification, tune β on validation data, starting with $\beta = 0.5$.

N.3 Formula Generation and Numerical Stability

Generated Python formulas undergo multi-stage validation to ensure correctness and numerical stability.

Sandboxed Execution. Formulas execute in a restricted environment with explicitly allowed and blocked operations:

- **Allowed:** Arithmetic ($+$, $-$, $*$, $/$), `np.clip`, `np.exp`, `np.log1p`, `np.maximum`, `np.minimum`, `np.abs`, `sigmoid`, feature access
- **Blocked:** File I/O, imports, system calls, loops, recursion, `eval`, `exec`

Numerical Stability Constraints. The decoder prompt includes explicit guidance:

Ensure numerical stability: (1) use `clip(x, min, max)` for bounded outputs; (2) add small constants to denominators (e.g., $x / (0.1 + y)$); (3) prefer `log1p(x)` over `log(x)` for values near zero; (4) bound exponential arguments to prevent overflow.

Post-Execution Validation. After execution, outputs are checked for:

- NaN or Inf values $\rightarrow$ formula rejected, re-generated with error message
- Values outside domain bounds $\rightarrow$ clipped with warning logged
- Type mismatches $\rightarrow$ formula rejected

Validation Statistics. Across all experiments: 94.2% of generated formulas passed validation on first attempt, 5.1% required one re-generation for syntax errors, 0.7% required re-generation for numerical issues, and 0% of final corrections produced invalid outputs on test data.

N.4 Prompt Templates

For reproducibility we reproduce the schema for text of every prompt used in MARICL. All prompts are templated; placeholders in `{curly braces}` are filled at runtime with the corresponding artifacts (high-residual examples, feature descriptions, prior critiques, etc.). No prompt is used at inference time — compiled formulas are executed directly in the sandbox of Appendix N.3.

Encoder prompts $\mathcal{P}_{\text{encoder}}^{(k)}$. The encoder analyses the augmented context $\mathcal{C}_{\text{aug}}$ and emits a structured hypothesis z_k . We use K templates that differ only in which residual aspect they emphasise (error patterns vs. direct sample patterns).

```
[ENCODER PROMPT #1 -- ERROR PATTERNS]
You are analysing systematic prediction errors of a base model.
Feature descriptions: {features}
Domain context: {domain_context}
High-residual examples (input, target, base-prediction, residual):
{high_residual_table}
Identify the feature combinations and nonlinearities that the base
model appears to miss. Output a structured hypothesis  $z_k$  with
fields:
- hypothesised_pattern: ...
- implicated_features: [...]
- functional_form_guess: ...
- rationale: ...

[ENCODER PROMPT #2 -- SAMPLE PATTERNS]
(implementation: MARICLPipeline._encode_latent_z_batched in
maicl_lib_v2.py, template encoder_with_data in maicl_config.yaml)

2. SAMPLE PATTERNS (direct pattern learning perspective):
- What direct relationships do you see between features and target
values in these samples?
- How do feature values relate to target values? (e.g., "When age is
high and BMI is high, target is typically high")
- What prediction rules would work based on the sample patterns
themselves?
- What feature combinations consistently lead to high/low target
values?
- Learn the TRUE UNDERLYING PATTERN from the samples, not just how to
fix errors.

IMPORTANT: Focus on learning to PREDICT based on sample patterns,
not just learning to fix specific errors.
Your mechanism should capture the true underlying relationships
visible in the samples.

[... full encoder_with_data template output requirements from
maicl_config.yaml ...]

Latent mechanism z:
```

Decoder prompt $\mathcal{P}_{\text{decoder}}$ (Eq. 8). The decoder converts z_k into the dual representation (T_k, f_k) .

```
[DECODER PROMPT]
Given the structured hypothesis below, produce (1) a
natural-language explanation  $T_k$  in 2-3 sentences, and (2) an
executable Python expression  $f_k$  using only the operators in
{allowed_operators}. Ensure numerical stability: clip bounded
outputs, add small constants to denominators, prefer log1p over
log near zero, and bound exponential arguments. Do not use loops,
imports, or file I/O.
Hypothesis: {z_k}
Return JSON: {"T_k": "...", "f_k": "..."}.
```

Critique prompt $\mathcal{P}_{\text{critique}}$ (Eq. 11). At iteration t , the critique prompt generates a textual gradient $g_k^{(t)}$ from the current state $S_k^{(t)}$ and the failure set $\mathcal{E}_k^{(t)}$.

```
[CRITIQUE PROMPT]
Current hypothesis: {z_k}
Current correction: {T_k, f_k}
Training loss: {L_k}
```

Worst remaining failures (input, target, current prediction, error):
{failure_set_table}
Diagnose why the correction fails on these examples. Distinguish structural mismatch (wrong functional form) from coefficient mismatch (right form, wrong scale). Suggest a concrete refinement to f_k that addresses the dominant failure mode without harming low-error examples.

Refinement prompt (Eq. 13). The refined correction at iteration $t+1$ is sampled conditional on the full textual state $S_k^{(t)}$ (which includes the critique $g_k^{(t)}$).

[REFINEMENT PROMPT]

(implementation: MARICLPipeline.decoder in maicl_lib_v2.py, template decoder_default in maicl_config.yaml)

At refinement time (iteration $t+1$), the {latent_z} placeholder is replaced by the accumulated state $S_k^{(t)}$ rather than $z_k^{(0)}$ alone.

You are a mechanism decoder that converts a latent mechanism representation into an interpretable, executable mechanism.

GOAL:

Produce mechanisms that are BOTH high-performing and highly interpretable, without unnecessary verbosity.

HARD REQUIREMENTS (do not skip):

1. The mechanism must be descriptive and textual (not just math).
2. The mechanism must explicitly include nonlinearities AND interactions.
3. The mechanism must introduce intermediate combinatory concepts (named, explained) that capture nonlinear interactions.
4. The final formula must be executable and appear as a SINGLE LINE starting with "Formula:" so it can be extracted programmatically.

WHAT TO INCLUDE:

- Named intermediate concepts (1-3): define them briefly, but do NOT rely on them in the final Formula line. Instead, inline/expand them in the final Formula expression.
- Nonlinear transforms: at least one (saturation/log/soft-threshold/inverse-U).
- Interaction terms: at least one (prefer nonlinear interaction form $x*y/(K+x*y)$).
- One short "why this helps" sentence referencing typical residual patterns (extremes / interactions / thresholds).

STYLE:

- Prefer causal language ("limits", "enhances", "inhibits", "gates", "synergizes") over purely statistical phrasing.
- Keep constants realistic and stable (avoid huge coefficients).
- Avoid full code blocks or function definitions; use compact math expressions.

Note on prediction-execution prompts. Earlier exploratory versions of MARICL used an LLM-based execution path (prompting the model to evaluate f_k on a query and return $\{y_{\text{hat}}, \text{confidence}\}$). This path is deprecated in the final framework: all reported numbers use direct sandboxed Python evaluation (Appendix N.3), so inference is zero-LLM-cost as stated in Section 2.4 and Appendix F. We retain no execution prompt in the released pipeline.

N.5 Data Preprocessing and Reproducibility

Feature Preprocessing. All numerical features are standardized to zero mean and unit variance using training set statistics, with identical transformations applied to validation and test sets. Categorical features are one-hot encoded.

Target Scaling. For regression tasks: Cell-Free Protein and Enzyme Activity targets are scaled to $[0, 1]$ via min-max normalization (reported MAE reflects scaled targets); Diabetes targets retain their original scale as a disease progression measure (R^2 is scale-invariant).

Dataset Sources and Licenses. Table 24 lists the provenance and license of every dataset used in this paper. All datasets are publicly redistributable under their stated terms; no proprietary or restricted data is used.

Table 24: Dataset sources, citations, and licenses.

Dataset	Source	Citation	License
Cell-Free Protein	Nature Comm. supp.	Borkowski et al. [2020]	CC BY 4.0
Enzyme Activity	UCI ML Repository	Dua, D. and Graff, C. [2019]	CC BY 4.0
Diabetes	scikit-learn / Efron et al.	Efron et al. [2004]	Public
Zoo	UCI ML Repository	Forsyth [1990]	CC BY 4.0
High School	UCI ML Repository	UCI Machine Learning Repository [2014]	CC BY 4.0
Adult Income	UCI ML Repository	Kohavi [1996]	CC BY 4.0
California Housing	TabArena / sklearn	Pace and Barry [1997]	CC0
Bike Sharing	TabArena / UCI	Fanaee-T and Gama [2014]	CC BY 4.0
Synthetic	—	—	MIT

Code and Library Licenses. All baselines are implemented using publicly released libraries under permissive licenses: scikit-learn (BSD-3), XGBoost (Apache 2.0), InterpretML/EBM (MIT), TabPFN (Apache 2.0 / model weights under prior-data-release terms; see PriorLabs repository), PySR (Apache 2.0), and SymbolicRegression.jl (Apache 2.0). LLM backbones are accessed via the official APIs of their providers (Google Gemini, OpenAI) under their respective terms of service; the open-source Llama-3-8B is used under the Meta Llama 3 Community License. Our own MARICL code will be released under the MIT license upon publication.

Baseline Implementations. All baselines use scikit-learn or official packages.

- Linear/Logistic Regression: `sklearn` with default L2 regularization.
- **XGBoost (tuned):** v1.7, 5-fold CV grid search on the training split, selecting by validation R^2 (regression) or macro F1 (classification). Grid: `n_estimators` $\in \{100, 200, 400, 600\}$, `max_depth` $\in \{3, 4, 6, 8\}$, `learning_rate` $\in \{0.03, 0.05, 0.1\}$, `reg_lambda` $\in \{0, 1, 2, 5\}$, `reg_alpha` $\in \{0, 0.1, 0.5\}$. For Adult Income we additionally sweep `scale_pos_weight` $\in \{1, 2, 3, 4\}$ to address class imbalance. Selected configurations and resulting metrics are reported in Table 25.
- EBM: `interpret` v0.4 with default settings.
- TabPFN: Official checkpoint (<https://github.com/PriorLabs/TabPFN>).
- Symbolic Regression: `PySR` v0.11, `niterations`=140, operators $\{+, -, *, /, \exp, \log, \sqrt{\cdot}, \sigma\}$, where $\sigma(x) = 1/(1 + \exp(-x))$ is provided as a custom unary operator via `PySR`’s `unary_operators` interface so that the synthetic benchmark’s planted nonlinearity is inside the search space; the same σ primitive is available to MARICL through its sandbox (Appendix N.3).
- `PySR` on residuals: same `PySR` configuration applied to $r_i = y_i - f_{\text{ML}}(\mathbf{x}_i)$.

Table 25: Tuned XGBoost vs. default XGBoost vs. XGBoost+MARICL across all benchmarks (mean $\pm$ std, 5 seeds; selection by 5-fold CV on the train split, reported on test). The headline MARICL+XGBoost vs. XGBoost gap narrows under tuning but persists across all nine benchmarks; on Cell-Free Protein and Adult Income the gap narrows from +0.144 R^2 and +0.108 macro F1 to +0.106 and +0.069 respectively. *Selected configurations (best CV winner per dataset)*: Cell-Free n=400, d=4, lr=0.05, $\lambda=1$, $\alpha=0.1$; Enzyme n=200, d=4, lr=0.05, $\lambda=2$, $\alpha=0$; Diabetes n=100, d=3, lr=0.05, $\lambda=2$, $\alpha=0.1$; California n=600, d=8, lr=0.05, $\lambda=1$, $\alpha=0$; Bike n=600, d=8, lr=0.03, $\lambda=1$, $\alpha=0$; Zoo n=200, d=4, lr=0.1, $\lambda=1$, $\alpha=0$; High School n=200, d=3, lr=0.1, $\lambda=2$, $\alpha=0.1$; Adult n=400, d=6, lr=0.05, $\lambda=2$, $\alpha=0.1$, scale_pos_weight=3.

Dataset	Metric	XGB (default)	XGB (tuned)	XGB+MARICL	Δ vs. tuned
Cell-Free Protein	R^2	0.579 $\pm$ 0.025	0.617 $\pm$ 0.022	0.723 $\pm$ 0.030	+0.106
Enzyme Activity	R^2	0.399 $\pm$ 0.020	0.428 $\pm$ 0.019	0.513 $\pm$ 0.010	+0.085
Diabetes	R^2	0.308 $\pm$ 0.015	0.354 $\pm$ 0.024	0.543 $\pm$ 0.025	+0.189
California Housing	R^2	0.832	0.844 $\pm$ 0.006	0.861	+0.017
Bike Sharing	R^2	0.894	0.903 $\pm$ 0.005	0.917	+0.014
Zoo	Acc	0.905 $\pm$ 0.030	0.914 $\pm$ 0.025	0.952 $\pm$ 0.025	+0.038
High School	Acc	0.517 $\pm$ 0.020	0.527 $\pm$ 0.020	0.540 $\pm$ 0.020	+0.013
Adult Income	Acc	0.813 $\pm$ 0.025	0.823 $\pm$ 0.006	0.832 $\pm$ 0.025	+0.009
Adult Income	macro F1	0.692 $\pm$ 0.025	0.731 $\pm$ 0.014	0.800 $\pm$ 0.025	+0.069